



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A MAJOR PROJECT PROGRESS REPORT
ON
CIRCUITRON: CIRCUIT RECOGNITION, TRANSFORMATION, AND ANALYSIS**

Submitted By:

Anveshan Timsina	(THA078BEI005)
Sandesh Dhital	(THA078BEI034)
Saroj Nagarkoti	(THA078BEI039)
Utsab Dahal	(THA078BEI046)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

February 2026



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
THAPATHALI CAMPUS**

**A MAJOR PROJECT PROGRESS REPORT
ON
CIRCUITRON: CIRCUIT RECOGNITION, TRANSFORMATION, AND ANALYSIS**

Submitted By:

Anveshan Timsina (THA078BEI005)
Sandesh Dhital (THA078BEI034)
Saroj Nagarkoti (THA078BEI039)
Utsab Dahal (THA078BEI046)

Submitted To:

Department of Electronics and Computer Engineering
Thapathali Campus
Kathmandu, Nepal

In partial fulfillment for the award of the
Bachelors Degree in Electronics, Communication and Information Engineering

Under the Supervision of

Er. Kobid Karkee

February 2026

ACKNOWLEDGEMENT

First, we would like to thank the Department of Electronics and Computer Engineering for providing us with an opportunity to learn and research on this project. We appreciate the department's assistance and recommendations in the selection and hopefully the successful completion of this project.

Special thanks to our supervisor, **Er. Kobid Karkee**, for his guidance, invaluable feedback, and constant support in regards to the project. His expertise and encouragement have been pivotal in shaping the direction, scope and implementation of this project. We are also grateful to our project coordinator, **Er. Sudip Rana**, for his continuous support and guidance.

Additionally, we would like to thank all of our teachers, friends and other direct and indirect contributors for their indispensable help, support and encouragement during our study and execution of the project. We are also grateful to the authors of various papers that we have referenced for our project.

Anveshan Timsina	(THA078BEI005)
Sandesh Dhital	(THA078BEI034)
Saroj Nagarkoti	(THA078BEI039)
Utsab Dahal	(THA078BEI046)

ABSTRACT

CIRCUITRON is an integrated system designed to bridge the gap between traditional hand-drawn electrical schematics and modern circuit simulation tools. This project consists of a robust pipeline that automates the transformation of handwritten circuit diagrams into SPICE-compatible netlists, enabling users to simulate, analyze, and interact with circuits in a digitally visualized and editable environment. Leveraging deep learning models such as YOLOv7 for component detection, custom OCR for text recognition, together with junction-based wire detection, the system has achieved a mAP@0.5 of 83% for component detection, and expects character accuracy of 75% for text recognition, and proper wire and node detection, for horizontal and vertical wires, across varied drawing styles. A user-friendly interface allows manual corrections, schematic editing, and direct integration with Ngspice for simulation. Additionally, the platform incorporates an LLM-powered assistant to answer circuit-related queries, thus reinforcing learning through natural language interaction.

Keywords: circuit recognition, object detection, SPICE netlist, text recognition, wire detection.

TABLE OF CONTENTS

ACKNOWLEDGEMENT	i
ABSTRACT	ii
LIST OF FIGURES	v
LIST OF TABLES	vi
1 Introduction	1
1.1 Background Information	1
1.2 Motivation	2
1.3 Problem Statement	2
1.4 Project Objectives	3
1.5 Project Scope and Applications	3
2 LITERATURE REVIEW	5
3 REQUIREMENTS ANALYSIS	8
3.1 Software Requirements	8
3.2 Feasibility Study	12
4 SYSTEM ARCHITECTURE AND METHODOLOGY	15
4.1 Block Diagram	15
4.2 Wire Detection Methodology	23
4.3 Terminal Detection	29
4.4 Line Detection via Path Finding	32
4.5 Use-Case Diagram	33
4.6 Performance Metrics Expectation	34
4.7 Error Handling	36
4.8 System Flowchart	41
4.9 Dataset Analysis	42
5 IMPLEMENTATION DETAILS	44
5.1 Dataset Manipulation and Preprocessing	44
5.2 Line Detection via Path Finding	50
5.3 Frontend and Backend Implementation	54
5.4 Circuit Simulation using PySpice	55
6 RESULTS AND ANALYSIS	57
6.1 Comparative Analysis of Object Detection using YOLOv5 and YOLOv7	57

6.2	Optical Character Recognition (OCR) Results	67
6.3	Junction-Based Wire Detection	71
6.4	Line Detection via Path Finding Visualization	78
6.5	Frontend Visualization	80
7	REMAINING TASKS	82
7.1	Summary of Remaining Tasks	82
8	APPENDICES	83
	Appendix A: Budget Estimation	83
	Appendix B: Gantt Chart	84
	REFERENCES	85

LIST OF FIGURES

Figure 4-1	Block Diagram of the CIRCUITRON System	15
Figure 4-2	Architecture of YOLOv7	17
Figure 4-3	Text Detection and Recognition Pipeline	19
Figure 4-4	Use-Case Diagram for CIRCUITRON System	34
Figure 4-5	System Flowchart	41
Figure 5-1	Interaction Diagram for the Web Application	55
Figure 6-1	Loss Curves for YOLOv5	57
Figure 6-2	Loss Curves for YOLOv7	58
Figure 6-3	Mean Average Precision (mAP)@0.5 vs Epochs for YOLOv5 and YOLOv7	58
Figure 6-4	mAP@0.5:0.95 vs Epochs for YOLOv5 and YOLOv7	59
Figure 6-5	Precision-Confidence Curve for YOLOv5	60
Figure 6-6	Precision-Confidence Curve for YOLOv7	60
Figure 6-7	Recall-Confidence Curve for YOLOv5	61
Figure 6-8	Recall-Confidence Curve for YOLOv7	62
Figure 6-9	F1 Score-Confidence Curve for YOLOv5	63
Figure 6-10	F1 Score-Confidence Curve for YOLOv7	63
Figure 6-11	Precision-Recall Curve for YOLOv5	64
Figure 6-12	Precision-Recall Curve for YOLOv7	65
Figure 6-13	Confusion Matrix for YOLOv5	66
Figure 6-14	Confusion Matrix for YOLOv7	66
Figure 6-15	Text Detection and Recognition (i)	68
Figure 6-16	Text Detection and Recognition (ii)	68
Figure 6-17	Training Loss vs Epochs (OCR)	69
Figure 6-18	OCR Model Predictions	70
Figure 6-19	OCR Performance Metrics and their Distributions	71
Figure 6-20	Otsu's Global Thresholding	72
Figure 6-21	Skeletonization	73
Figure 6-22	Component Removal	74
Figure 6-23	Wire Classification	75
Figure 6-24	Segment Consolidation Through Merging	76
Figure 6-25	Bounding Box Overlay	77
Figure 6-26	Wire-Component Intersection Detection	78
Figure 6-27	Skeletonized image with detected endpoints	79
Figure 6-28	Adjacency graph constructed using Multi-Head BFS algorithm	80
Figure 6-29	Digitally Drawn Circuit	81
Figure 6-30	Example Circuit Simulation Result	81

LIST OF TABLES

Table 4-1 Expected Performance Metric Values for Individual Modules	34
Table 4-2 Summary of Data in the CGHD Dataset	42
Table 4-3 Frequency of Each Class with its Code in the CGHD Dataset	43
Table 5-1 Time complexity breakdown for Multi-Head BFS algorithm	53
Table 5-2 Space complexity breakdown for Multi-Head BFS algorithm	54
Table 6-1 Comparison of Performance Metrics for YOLOv5 and YOLOv7	67
Table 7-1 Summary of Remaining Tasks	82
Table 8-1 Project Budget Breakdown	83
Table 8-2 Project Gantt Chart	84

LIST OF ABBREVIATIONS

AC	Alternating Current
ADC	Analog-to-Digital Converter
AI	Artificial Intelligence
AP	Average Precision
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
BJT	Bipolar Junction Transistor
CGHD	Circuit Graph Handwritten Diagrams
CLI	Command Line Interface
CNN	Convolutional Neural Network
CRNN	Convolutional Recurrent Neural Network
CSV	Comma-Separated Values
CTC	Connectionist Temporal Classification
DAC	Digital-to-Analog Converter
DC	Direct Current
DL	Deep Learning
DOM	Document Object Model
EDA	Electronic Design Automation
ELAN	Efficient Layer Aggregation Network
FET	Field-Effect Transistor
FSM	Finite State Machine
GNN	Graph Neural Network
GPU	Graphics Processing Unit
GUI	Graphical User Interface
HCD	Handwritten Circuit Diagram
HSV	Hue, Saturation, Value
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
IEEE	Institute of Electrical and Electronics Engineers
IoU	Intersection over Union
JSON	JavaScript Object Notation
LLM	Large Language Model
LMDB	Lightning Memory-Mapped Database
LSTM	Long Short-Term Memory
mAP	Mean Average Precision
ML	Machine Learning
NLP	Natural Language Processing

NMS	Non-Maximum Suppression
OCR	Optical Character Recognition
PAN	Path Aggregation Network
PCB	Printed Circuit Board
PHT	Probabilistic Hough Transform
RC	Resistor-Capacitor
RCNN	Region-based Convolutional Neural Network
RF	Radio Frequency
RMS	Root Mean Square
SGD	Stochastic Gradient Descent
SPICE	Simulation Program with Integrated Circuit Emphasis
VQA	Visual Question Answering
YOLO	You Only Look Once

1 INTRODUCTION

“CIRCUITRON: Circuit Recognition, Transformation, and Analysis” is a system to convert hand-drawn electrical circuit diagrams into Simulation Program with Integrated Circuit Emphasis (SPICE) compatible netlists enabling simulation and graphical analysis along with digital visualization of the circuit in an editable schematic. The system is also to integrate a prompt-based Artificial Intelligence (AI) assistant to help answer queries related to the relevant circuit.

1.1 Background Information

The first step of learning about electrical and electronic circuits involves looking at and hand-drawing said circuits with the necessary components. These hand-drawn circuit diagrams continue to play an important role in electronics education and early-stage hardware design due to their ease of use, speed, and accessibility. However, such hand-drawn diagrams prove to be limited in their utility in the current education landscape that prioritizes intuitive and visual understanding as they require significant mental gymnastics to facilitate any form of analysis of the circuits, especially when the change in parameters is incorporated.

A slew of rather easy-to-use circuit simulation and analysis tools such as `ngspice` and KiCad exist, of course; and they continue to foster visual learning and analysis of circuits of a range of complexities. But these traditional Electronic Design Automation (EDA) tools lack the capability to accept hand-sketched circuit diagrams as input. Instead, users are required to redraw the circuits in software using Graphical User Interface (GUI) tools or by manually entering SPICE netlists; steps that are both tedious and error-prone [1, 2]. Be it due to the (admittedly not very steep yet inconvenient) learning curve of these tools or just the additional effort of having to redraw a circuit digitally (the dullness of which only goes up with circuit complexity), the instructors as well as the students often tend to neglect arguably essential simulation-based intuitive teaching and learning. This again encourages the same long-established rote learning practice instead of ushering in a new and perceptively intuitive way of understanding circuits and electronics at large. And it has been long proven that students tend to perform better when given the appropriate simulative manipulation and concept clarification tools, especially in the context of electrical and electronics circuits [3]. It then follows that the reluctance to use such tools is actively hindering the students’ potential. Thus, the use of visual manipulative simulation tools is the need of the hour. In fact, it has been the need of the hour for quite a while. But, due to a lack of any uber-convenient method, it is yet to be popularly embraced, primarily in technologically nascent countries such as Nepal.

1.2 Motivation

In recent years, the mantra of the modern world has seemingly become “adapt or die”. The society regularly stomps on the tech-illiterate to make way for the ones willing to champion the use of new and shiny tools in all aspects of their lives. In an increasingly globalized economy, this is especially true for the up-and-coming students who need to compete with other students all over the world relying solely on their understanding of subject matter in their particular fields. In such case, it becomes essential that they are given the tools to enhance their learning and understanding even by minor increments. In the context of electronics, circuits are the fundamental building blocks that permeate each and every aspect of our “teched-out” world and it is important for learners in the field to have an intuitively in-depth understanding of this fundamental ingredient. Thus, there exists a motivation for a hassle-free method to analyze and understand circuits to deepen the necessary conception and comprehension.

The motivation stems not only from a deep-seated commitment to give the instructors and students tools to easily visualize and analyze circuits in the belief that it will genuinely improve the quality of professionals and engineers born out of universities, but also from the personal experience of the members involved in this project. As the electronics engineers of the near future who have had and will continue to have to deal with circuits in one form or another, the members involved in this project are uniquely positioned to understand the plight of a beginner in the study of electronics. And a tool which can convert a hand-drawn circuit into a digital schematic and also allow for analysis would go a long way in alleviating that plight.

1.3 Problem Statement

The EDA tools currently available do not support the direct transformation of hand-drawn circuit images into simulation-ready schematics. There exist a few nascent prototypes for the same but there seems to be no widely adopted, robust, and user-friendly pipeline that combines component recognition, reliable wire connectivity, schematic generation and simulation, whether by itself or by integration with SPICE simulators.

“CIRCUITRON: Circuit Recognition, Transformation, and Analysis” project aims to create an integrated system capable of first, recognizing and digitizing hand-drawn circuits into editable schematics and then allowing the user to analyze those circuits through graphs using SPICE-compatible netlists with an added AI assistant to handle related queries.

1.4 Project Objectives

- To automatically convert hand-drawn circuit diagrams into SPICE-compatible netlists for simulation.
- To visualize the circuit in an editable schematic in addition to an AI assistant for handling simple, relevant queries.

1.5 Project Scope and Applications

1.5.1 Project Scope

This project focuses on the development of a software pipeline that accepts scanned or photographed images of hand-drawn electrical circuits and transforms them into SPICE-compatible netlists for simulation as well as editable schematics through an intermediate integrated data structure to facilitate the transformation. The scope includes detecting common circuit components and their values in mentioned units, inferring wire connectivity with junction recognition and generating a circuit diagram with simulation capabilities. Additionally, the system is to incorporate an Large Language Model (LLM)-based query-solving AI assistant. The system will support basic circuits and components but will exclude any form of advanced circuit transformation and analysis such as in circuits associated with Radio Frequency (RF), mixed-signal, or Printed Circuit Board (PCB) designs.

1.5.2 Project Applications

- **Educational utility:** The system is foremost, to be an educational tool designed to assist students and instructors in conveniently digitizing circuit diagrams and understanding circuit behavior through limited simulation and interactive Q&A.
- **Lightweight prototyping:** The project is to enable rapid comparison of hand-drawn designs with SPICE simulation results early in the design process for quick feedback and validation.
- **Mobile simulation tool:** The project in its final form can facilitate quick on-the-go analysis of circuits for students and potentially, field engineers as well.

1.5.3 Project Limitations

Despite its innovative concept, the system is constrained by several factors. First, its accuracy depends heavily on the quality and clarity of input images; low-resolution or messy sketches may lead to recognition errors. The fix for this would obviously be more rigorous training on appropriate training data to chip away at the error but finding the ideal dataset with enough examples for all kinds of components and circuits has proven

challenging. Additionally, the system is to support only a fixed set of commonly used components and may fail on rare or user-defined symbols. Wire connectivity assumes standard drawing practices however there is no one way to draw a circuit diagram; non-standard or overlapping wires as well as complicated circuits may confuse the parser. Moreover, the system is to lack a simulation engine of its own and instead use an open-source tool like Ngspice, which not only affects the seamlessness of the user experience but also limits the simulation capabilities to that of Ngspice. For instance, Ngspice needs SPICE models to be imported for complicated circuit elements such as Analog-to-Digital Converter (ADC), Digital-to-Analog Converter (DAC), timers, etc. This means even if the system can recognize these hand-drawn circuit elements, it will not be able to simulate their functionality.

2 LITERATURE REVIEW

Hand-drawn circuit recognition and digitization, while not a vastly researched area, has a few notable examples of its implementation. Most of the study and implementation related to this has been done on classifying the different circuit components (like battery, resistors, inductors, etc.) and there's not much on arguably the more difficult aspect of circuit recognition: node and wire connectivity. For instance, in a 2021 study, 20 different components were classified from self-made hand-drawn circuits with the recognition accuracy of 97.33% [4].

In one of the oldest studies on the automatic recognition of hand-drawn circuits, an approach of identifying the components first and then the node segments (taking into consideration their spatial relationship) was implemented, achieving an accuracy of 86% for components and 92.5% for the nodes. The node segments were then traced assuming wire between them, considering a variety of hard-coded conditions [5]. In a recent 2024 research, a couple of engineers were successful in digitizing images of electrical circuit schematics using a novel pattern-recognition methodology and custom-trained OCR model achieving respective success rates of 86.4% and 99.6% [6]. This, however, was trained on printed/computer-made circuit diagrams, not hand-drawn ones and also isn't able to understand the values of the circuit components.

To overcome the limitations of Convolutional Neural Networks (CNNs) in understanding circuit topology, graph-based methods have gained traction. Circuit2Graph, a pipeline that transforms circuit diagrams into graph structures where nodes represent components and edges denote electrical connections was introduced in 2023. Using Graph Neural Networks (GNNs), their system achieved 97.9% accuracy on circuit classification tasks, outperforming traditional methods [7]. This approach demonstrated that GNNs can effectively model the global structure of a circuit, even when visual features alone are insufficient. Similarly, Bayer et al. [8] used Mask Region-based Convolutional Neural Network (RCNN) for instance segmentation of components in hand-drawn diagrams, followed by graph extraction techniques to infer connections. Their model achieved around 94% mask accuracy, but they noted that symbol recognition was "reasonable, yet incomplete," especially in more complex drawings or where handwriting was less consistent. A paper published in the International Research Journal of Engineering and Technology proposed a two-step method for electronic circuit diagram detection and recognition; first detecting and classifying the components present in the circuit and then labeling the classified components to form a string which is then fed to an Finite State Machine (FSM) system for circuit recognition [9].

With the use of YOLOv5 for detection of circuit components and Probabilistic Hough transform-based solution for node recognition and taking in hand-drawn circuits, a

couple of Indian engineers were successful in achieving not only a mAP of 98.2% in component detection but also a near-real time generation of circuit schematic with an accuracy of 80% [1]. However, their circuits were not labelled which is to say the values for the battery (in volts), resistance (in Ohms), etc. were not written in the circuit. Another very recent study suggested a three-step process to detect and identify the circuit and its components with the steps including detection of components, detection of lines and their junctions and then the detection of text placed near the components and then combining the data from these three streams into a JavaScript Object Notation (JSON) format to eventually derive a netlist from [10]. This is a similar workflow to the one this project intends to use; however, the scope of our project is much larger owing to the fact that an editable schematic is to be delivered on top of taking in hand-written circuit diagrams as opposed to generating a netlist from a digital schematic.

Netlist generation is also a very critical aspect of this domain. While traditional EDA tools such as KiCad and support netlist export from manually created schematics, they do not allow direct image-to-netlist conversion [11]. This limitation was addressed with Img2Sim; an artificial neural network-based system that detects components, infers connectivity, and outputs a complete SPICE netlist. Their model reported over 90% accuracy in topology extraction and represents one of the first functional pipelines to translate a hand-drawn circuit into a ready-to-simulate format [2]. However, Img2Sim and similar systems typically require clean input images and are limited to a predefined set of components, reducing their robustness in real-world use cases. While the feasibility of such pipelines is now evident, a generalized, accurate, and open-source solution that goes the extra mile to also visualize the circuits digitally remains unavailable.

Parallel to recognition and netlist generation, visual question answering (Visual Question Answering (VQA)) for circuits is emerging as a useful educational application. CircuitVQA is a dataset containing more than 115,000 question-image pairs based on 5,725 unique circuits. It covers approximately 70 symbol types and supports the training of transformer-based VQA models capable of answering questions like “What is the function of R1?” or “Is this a voltage divider?” Early experiments demonstrated that models fine-tuned on CircuitVQA outperformed generic VQA systems, indicating the importance of domain-specific training. Meanwhile, large language models (LLMs) such as GPT-4 have also been explored for circuit explanation tasks. For instance, the SPICEPilot framework attempts to generate and interpret SPICE code using GPT-based models [12]. However, these general-purpose models often fail to respect domain conventions, such as valid W/L ratios or simulation types, resulting in inaccurate outputs.

Finally, simulation integration plays a vital role in validating and interacting with generated circuits. Ngspice, an open-source simulation engine, is widely used in

research and is natively supported in KiCad. It enables Direct Current (DC), Alternating Current (AC), and transient analysis, making it suitable for most educational and prototyping needs. In contrast, commercial tools like Simulink and PSpice provide broader capabilities but are cost-prohibitive. Some research efforts, including Img2Sim, have leveraged open standards to generate SPICE files compatible with tools like Ngspice or PSpice. Other projects utilize Python-based libraries such as PySpice to enable automated simulation directly within a code-driven environment which is exactly how this project intends to accomplish its simulation needs.

While significant progress has been made in component recognition and value comprehension, connectivity inference, netlist generation, and interactive Q&A, existing systems remain fragmented or limited in scope. An end-to-end system that combines all these elements into a user-friendly platform remains a critical need for both educational and practical applications.

3 REQUIREMENTS ANALYSIS

3.1 Software Requirements

3.1.1 Python Libraries

Programming Language is the heart of the machine learning aspects of the project as well as parts of the backend. with its suite of various packages and libraries is used to facilitate the implementation of models for object detection, text recognition, and even simulation visualization.

- **NgSpice and PySpice:** After detecting and extracting relevant circuit elements, the project aims to simulate the circuit. is the open-source SPICE simulator for electric and electronic circuits which takes in netlists and enables the simulation and analysis of circuits. And, is a library that provides bindings to to build and simulate circuits entirely within .
- **PyTorch and TensorFlow:** PyTorch and TensorFlow, both Deep Learning (DL) libraries, which serve as the backbone for training and deploying machine learning models used in the project pipeline. They are used to first integrate existing models, specifically YOLOv7 and OCR, into the project and then also fine-tune and train them depending on the need. Also, the seamless integration of these libraries with other widely used libraries, including NumPy and Pandas, streamlines data preprocessing and manipulation. Both PyTorch and TensorFlow, while having different strengths, serve similar purposes for this project and use of one over the other just comes down to personal preference and convenience.
- **Numpy:** Numpy is an open-source library for all tasks related to numerical computing and data manipulation. CIRCUITRON uses Numpy for array manipulation and pixel-level operations on images. Numpy seamlessly integrates with OpenCV and helps streamline the fundamental processes of image processing like converting images into matrices, performing image-wide as well as element-wise operations important for augmentation, thresholding and segmentation.
- **OpenCV:** OpenCV is the primary library for pre-processing the circuit images and performing low-level computer vision tasks. It is used in almost all aspects of image processing in the project like thresholding, edge detection, geometric augmentations, contour detection, morphological operations (dilations, erosion) as needed. These operations help in extracting meaningful visual information before feeding the data into the models.
- **Pandas:** Pandas provides data structures, such as data frames, that allow for efficient handling and manipulation of structured data. It offers functions for reading and writing data from various file formats, data cleaning and preprocessing, filtering and

selecting data, grouping and aggregating data, and more. Pandas is used in the project to organize structured data related to the circuit components and their properties which includes storing component lists with attributes like label, value, position, and orientation, managing the intermediate data between detection, parsing, and netlist generation, and exporting tables or logs for debugging or evaluation.

- **Matplotlib and Plotly:** Matplotlib is a widely used data visualization library in . Matplotlib is used in the project for visualizing the performance metric scores in graphs. Furthermore, it is to be used for plotting simulation outputs with integration with . Plotly is another library for data visualization which supports interactive diagrams (handy for showing simulation graphs in motion).
- **Scikit-learn:** In context of the project, Scikit-learn is used for post-processing tasks such as clustering junction points, filtering outliers, or classifying ambiguous component types after initial detection. It includes tools for data splitting, metrics, and evaluation which are extremely useful during model development and benchmarking. Additionally, it can assist in component matching, value prediction, or validation models if required.
- **Seaborn:** Seaborn is a data visualization library based on Matplotlib, designed for creating statistical graphics with attractive default styles. It is used to visualize distributions, such as accuracy of component detection, or OCR confidence scores. It is essentially a wrapper for Matplotlib which makes working with graphs more convenient. It can also be used with Pandas to plot trends in simulation outcomes (say, voltage over time across components).
- **Hugging Face:** Hugging Face provides a large hub of pretrained transformer models for Natural Language Processing (NLP), vision, and more, along with the transformers library. It will be used in the project to integrate LLMs for answering circuit-related queries.
- **LangChain:** LangChain is a framework that helps build applications powered by LLMs, focusing on prompt engineering, chaining logic, memory, and tool use. It will be used to build the LLM-based AI assistant for the user interface by integrating circuit data (netlist, components, simulation results, schematics) to provide context-aware answers.
- **LangGraph:** LangGraph extends LangChain to allow for multi-agent or multi-step workflows using graph-like state machines. It can be used to build structured conversational workflows where the AI assistant interacts with different parts of the circuit data pipeline while maintaining session history.

3.1.2 YOLOv7

The YOLOv7 model is used to detect and classify circuit components in hand-drawn circuit images. It outputs bounding boxes, class labels, and confidence scores for each detected component. YOLOv7 is selected due to its superior detection accuracy, efficient training capabilities, and support for multiple object scales through its enhanced architecture. Compared to previous versions like YOLOv5, YOLOv7 delivers better performance in terms of mAP and inference speed.

3.1.3 DTRB Framework-Based OCR

A Convolutional Recurrent Neural Network (CRNN)-based OCR model trained from scratch, following the DTRB design philosophy for recognizing text is vital for the project as it is the core technology which reads and understands the component values and units of circuit elements. This was chosen for the project because, foremost, other ready-made OCR models failed to perform well even after some fine-tuning.

3.1.4 KiCad Symbol Library

KiCad is a free, open-source EDA suite which offers its libraries to be used freely to build any educational project. The project intends to use the KiCad symbols library, specifically, in order to simply extract the component symbols standardized by KiCad rather than building the icons/representations for components from scratch for use in the frontend schematic.

3.1.5 Coding and Training Platforms

- **Google Colab:** Google Colab is a cloud-based platform that offers a Jupyter notebook environment for developing deep learning systems. For this project, Google Colab provided with Graphics Processing Units (GPUs) for free, which significantly accelerated the training process, enabling experimentation with larger models and datasets. Colab also comes pre-installed with popular deep learning frameworks such as TensorFlow and PyTorch, widely used for Machine Learning (ML)-related tasks. Leveraging these frameworks, YOLO (v5 and v7) and OCR were implemented and experimented on.
- **Kaggle:** Kaggle is a multi-faceted online platform popular for data science competitions, datasets, code sharing and model training. It was primarily used to find relevant datasets (Circuit Graph Handwritten Diagrams (CGHD), VQA, Handwritten Circuit Diagram (HCD)), code sharing and model training. The dataset of choice for this project was also found in Kaggle. Additionally, it provided a fully hosted environment with popular libraries such as Pandas, Numpy, Tensorflow, PyTorch pre-installed to make implementing and training models convenient, especially with

its integration with Google Colab. Its free cloud computing service providing 30 hours of GPU use per week proved indispensable in accelerating the training and testing of different models.

- **Lightning AI:** Lightning AI is a cloud workspace, similar to Google Colab and Kaggle, which specializes in helping build agents and train models without any tedious setup. Most importantly, through the use of a student account, it allows for the use of extremely powerful GPUs for free (for a reasonable amount of time). Among all the cloud computing services used for the project, Lightning AI proved the most invaluable for implementing and training the models as the GPUs it allotted were super-fast and saved a ton of time during the tedious and time-consuming process.
- **Visual Studio Code:** The local Integrated Development Environment (IDE) of choice for this project is VS Code due to its extensive features auto completion, syntax highlighting, and copilot integration. Additional features are further integrated by the use of a suite of easy-to-use extensions which are tailored for each individual project member in their setup as needed. All the code to be run locally is run on VS Code and further down the line, its utility is expected to increase ten-fold during the web development phase.

3.1.6 Web Development

Although the web development part of this project has not kicked into gear as of yet; following are the tools and technologies expected to be used in the process:

- **Reactjs:** Reactjs will serve as an integral technology underlying the dynamic front-end interfaces to be developed for the project. The frontend for this project is expected to include dynamic elements like dragging, erasing, resizing, etc. of the circuits and circuit elements, and Reactjs is expected to fulfill these requirements with convenience and speed through the use of its features like virtual Document Object Model (DOM) abstraction, distinction of code into reusable logic and presentation buckets, etc. as well as compatible packages like react-draggable.
- **Next.js:** Next.js is a React-based web framework that enables the development of fast, scalable, and server-rendered web applications. It provides features such as routing, server-side rendering, and Application Programming Interface (API) integration, which are essential for building a responsive and user-friendly frontend. In this project, Next.js is used to develop the client-side interface, enabling efficient interaction with the backend services and smooth visualization of recognition results.
- **Chart.js:** Chart.js is a lightweight JavaScript library used for creating interactive and visually appealing charts and graphs in web applications. Chart.js is used to visualize analytical results such as performance metrics, recognition accuracy, confidence scores,

and statistical summaries on top of the graphs generated after simulation in a clear and intuitive manner. Its ease of integration with modern frontend frameworks and support for real-time data updates make it suitable for presenting system outputs effectively.

- **FastAPI:** FastAPI is a modern, high-performance web framework used for building RESTful APIs. It is designed for rapid development, high execution speed, and automatic generation of interactive API documentation. In this project, FastAPI is used to implement the backend services that handle requests from the frontend, manage data exchange, and interface with the text recognition and analysis modules. Its native support for asynchronous operations and integration with Pydantic ensures efficient request handling and robust data validation.
- **Uvicorn:** Uvicorn is a high-performance Asynchronous Server Gateway Interface (ASGI) server used to run asynchronous web applications. It is commonly used to serve FastAPI-based backend services due to its low latency and efficient handling of concurrent requests. In this project, Uvicorn is responsible for hosting the backend API, ensuring fast communication between the frontend application and the OCR processing modules.
- **Pydantic:** Pydantic is a library used for data validation and schema definition using type annotations. It ensures that data exchanged between the frontend and backend conforms to predefined structures and types. In this project, Pydantic is used to define request and response models, improving data integrity, and reducing runtime errors.
- **MongoDB:** MongoDB is a NoSQL database which stores data in JSON documents organized into collections. It is to serve the database needs for this project which enables functionalities like saving a digital schematic with its component values, simulation result storage, etc.
- **Github:** Github is a platform to store, manage and share code with integrated version control. Github enabled effective sharing and backing up of all the code involved in this project as of yet and allowed for all the participants to be on the same page at any given time. Github is expected to be used extensively throughout the project.

3.2 Feasibility Study

3.2.1 Technical Feasibility

Technical Feasibility assesses whether the project can be developed using available resources and expertise. The system leverages well-established technologies such as , OpenCV, custom OCR, YOLOv7, Ngspice, FastAPI, Next.js, all of which are stable and widely supported. For component detection, YOLOv7 provides effective object recognition, making it suitable for identifying handwritten circuit symbols. Wire and node detection rely on proven techniques like skeletonization, junction-based wire

detection, and clustering (e.g., K-means). Optical Character Recognition (OCR) is handled effectively using custom-trained OCR, ensuring accuracy even with noisy text. Simulation is facilitated through Ngspice Command Line Interface (CLI) in tandem with Pyspice. The team consists of skilled electronics and software engineers proficient in these technologies. The primary technical risk involves processing low-quality or cluttered circuit images, but the system is to include manual correction features to mitigate this. Overall, the project is technically feasible, with all necessary tools available and a well-matched skill set.

3.2.2 Economic Feasibility

Economic Feasibility examines whether the project is cost-effective and offers a good return on investment. The estimated costs include NPR 20,000 for computation (Google Colab Pro and cloud GPUs), NPR 4,000 for hosting and storage (domain, cloud storage, and database), NPR 2,160 for Institute of Electrical and Electronics Engineers (IEEE) membership (access to research journals), and NPR 15,000 for miscellaneous expenses (printing, presentations, memberships, etc.). Data collection and software licenses incur no cost since open-source datasets and tools are utilized. The total estimated budget is approximately NPR 41,160. Given its potential as an educational tool, the project could even be commercialized for academic institutions. The project is considered economically feasible due to its low development cost for the value it has to the education field.

3.2.3 Operational Feasibility

Operational Feasibility evaluates whether the system will function effectively in real-world scenarios. The tool addresses a genuine need by simplifying the conversion of hand-drawn circuits into simulations, eliminating manual redrawing. Its user-friendly interface allows optional manual edits to improve reliability, and cross-platform accessibility is ensured through a web-based (Next.js and FastAPI). Minimal training is required due to its intuitive design, and performance is optimized with near real-time processing using YOLOv7 and efficient OCR techniques. The modular architecture ensures easy maintenance, and the system can be scaled to support additional features in the future. The project is considered operationally feasible.

3.2.4 Time Feasibility

The project has to be developed in two sequential phases, with Phase A focusing on implementing the core functionality including the circuit recognition and text extraction engine, and netlist generation within a basic web framework. This phase has been successfully completed and tested.

Phase B is supposed to expand the system the system by developing the full web application, integrating NgSpice for simulations with a user-friendly UI to display and edit schematic diagrams, and implementing the AI question-answering feature; all while maintaining consistency with the pre-designed wireframes. A two-week buffer period had been planned between phases to accommodate thorough testing and necessary adjustments, with Phase B work commencing only after Phase A components have been fully validated. This structured approach ensures the maintenance of design integrity while progressively enabling the deliverance of more advanced features. As of yet, phase A has been completed and phase B is also nearly done, and advancing at a reasonable pace. The project is considered feasible from a time perspective.

4 SYSTEM ARCHITECTURE AND METHODOLOGY

4.1 Block Diagram

The process of converting a handwritten circuit diagram into a structured netlist involves a series of coordinated steps, each step feeding into the next, designed to detect, extract, and represent pictorial information in a machine-readable format and then using this data for simulation and schematic generation.

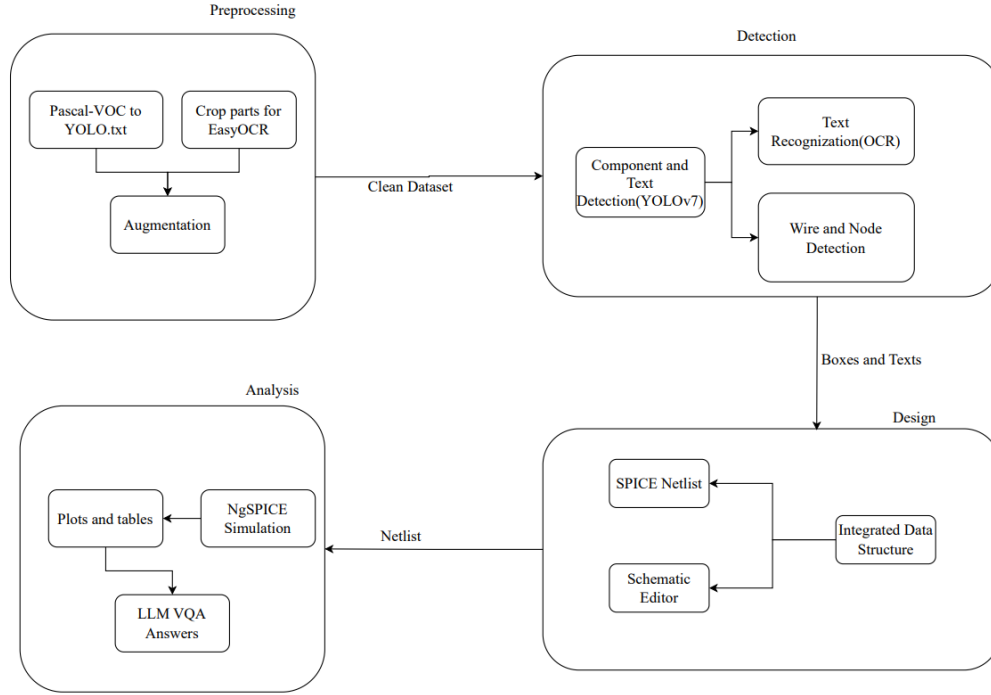


Figure 4-1 Block Diagram of the CIRCUITRON System

The high-level architecture of the CIRCUITRON system is visualized in the block diagram in Figure 4-1. Each step involved in the hand-drawn image to simulation-ready schematic pipeline is elaborated as follows.

4.1.1 Preprocessing

At first, the circuit information (components, component values, units, wire intersection, etc.) needs to be extracted out from the hand-drawn image. Appropriately fine-tuned ML and DL models are to be used for each of these steps which will be elaborated on in later subsections, however, for any sort of detection to happen, the image needs to first be pre-processed. Preprocessing for this not only includes image manipulation tasks but also conversion of the dataset annotations into appropriate formats to maintain compatibility with the component detection and text recognition models. The dataset of choice is in the Pascal VOC format which is incompatible with the models that are

intended to be used in the pipeline and hence need to be converted. The code for this conversion with examples will be discussed in the implementation section.

Furthermore, image preprocessing, which includes steps like denoising, contrast and brightness enhancement (for improved clarity) is a vital step in readying the dataset for training any and all models. However, geometric augmentations, especially rotation, is not appropriate for this project.

Among the various available preprocessing techniques, the ones to use are to be selected with careful consideration of the type of images available in the dataset as well as the models in use. Besides image manipulation using various techniques, preprocessing also includes splitting the dataset into training, validation and testing subsets as appropriate.

4.1.2 Circuit Component Detection

A critical step in the pipeline is the accurate detection of circuit components from hand-drawn schematic images. This task is addressed using a real-time object detection algorithm capable of identifying and localizing multiple electronic components (like resistors, capacitors, voltage sources, etc.) directly within the image. YOLO algorithm, the YOLOv7 model, specifically, has been selected for the component detection needs of this project. It is to be noted that text, as a generic collection of letters, numbers, and special characters is also considered a component in this context.

You Only Look Once (YOLO) (You Only Look Once) is a real-time object detection algorithm known for speed and efficiency. YOLO frames object detection as a single regression problem, straight from image pixels to bounding box coordinates and class probabilities. This model works as follows: first, dividing the image into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose centre falls within it. For each cell, a number of bounding boxes and their corresponding confidence scores (which represents the accuracy of the box and whether the box contains an object) are predicted. Each predicted bounding box includes the coordinates of the box centre relative to grid cell, represented by (x, y) , the width and height relative to the whole image, represented by (w, h) and the confidence score C given as:

$$C = P(\text{object}) \cdot \text{IoU} \quad (4-1)$$

where $P(\text{object})$ denotes the probability of an object being inside the box, and Intersection over Union (IoU) is the Intersection over Union between the predicted box and the ground truth box.

Additionally, for each box, class probabilities are predicted as:

$$\text{Class Score} = P(\text{class } i \mid \text{object}) \cdot C \quad (4-2)$$

This allows the model to classify which object exists in each box and once the predictions are made, non-maximum suppression (Non-Maximum Suppression (NMS)) is applied to discard less-relevant overlapping boxes.

Based on similar fundamental working principle, several iterations (versions) of YOLO have been introduced and popularized among which YOLOv7 is the YOLO model of choice for this project.

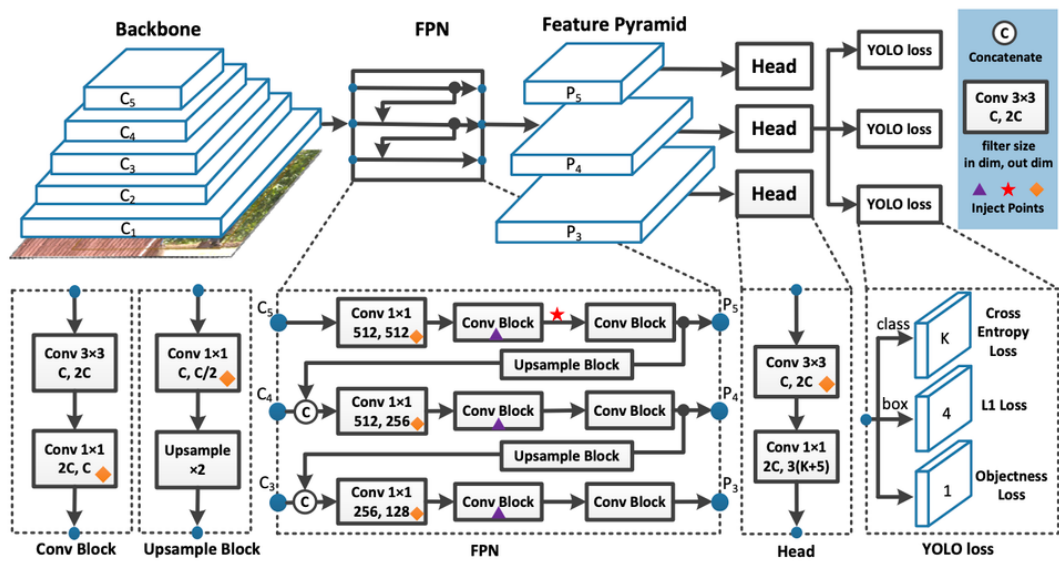


Figure 4-2 Architecture of YOLOv7

The architecture of the YOLOv7 model is composed of three primary modules: Backbone, Neck, and Head.

- **Backbone:** YOLOv7 uses an extended version of Efficient Layer Aggregation Network (ELAN) (Efficient Layer Aggregation Network) as its backbone. This structure enables deeper network designs without significantly increasing computational cost. The backbone is responsible for extracting hierarchical features from the input image while preserving gradient flow and feature reuse.
- **Neck:** The neck in YOLOv7 incorporates a Path Aggregation Network (Path Aggregation Network (PAN)) and ELAN-W modules. PAN enhances feature fusion across different scales, improving detection accuracy for both small and large objects. ELAN-W further refines spatial and contextual information during feature aggregation.
- **Head:** The head predicts object classes, bounding box coordinates (x, y, h, w), and objectness scores. YOLOv7 uses a decoupled head, which separates classification

and regression branches for better convergence and performance. The model outputs predictions at multiple scales to capture objects of varying sizes.

Further features of the model are as follows:

- **Bounding Box Prediction:** The prediction of bounding box coordinates in YOLOv7 is guided by anchor-based regression, where the final bounding box $B = (x, y, w, h)$ is computed relative to the anchor box using the following formula:

$$x = \sigma(t_x) + c_x \quad (4-3)$$

$$y = \sigma(t_y) + c_y \quad (4-4)$$

$$w = p_w \cdot e^{t_w} \quad (4-5)$$

$$h = p_h \cdot e^{t_h} \quad (4-6)$$

where (t_x, t_y, t_w, t_h) are the predicted offsets, (c_x, c_y) is the top-left corner of the grid cell, and (p_w, p_h) are the dimensions of the anchor box.

- **Training Enhancements:** YOLOv7 introduces Coarse-to-Fine Lead Head, Model Scaling, and Auxiliary Heads that help improve gradient flow and stability during training. It supports Label Assignment Strategies such as dynamic-k matching for optimal anchor-label association.
- **Data Augmentation:** YOLOv7 applies automatic data augmentation during training including mosaic augmentation (combining four images into one), Hue, Saturation, Value (HSV) color augmentation, and random flipping, cropping, and scaling. These help increase dataset diversity and improve generalization.
- **Early Stopping and Training Stability:** YOLOv7 supports built-in early stopping based on a patience threshold if validation loss (combining box loss, objectness loss, and classification loss) does not improve for several epochs. This ensures faster convergence and prevents overfitting.

4.1.3 Recognition of Component Values

In order to generate a complete and simulation-ready netlist, it is essential to not only detect the components themselves but also to extract their associated parameter values, which are generally annotated near the components in the circuit diagram. Thus, it becomes essential to integrate Optical Character Recognition (OCR) with the

object detection pipeline to extract and map these values correctly to the corresponding components.

This involves identifying regions within the circuit image that likely contain text (done by YOLOv7), and then recognizing said texts in order to parse the component values along with their units.

The hand-written text recognition is done using a CRNN-based OCR system, following the design philosophy of Deep Text Recognition Benchmark (DTRB) framework, however minor changes deemed suitable for this project have been made to get the text detection and recognition pipeline.

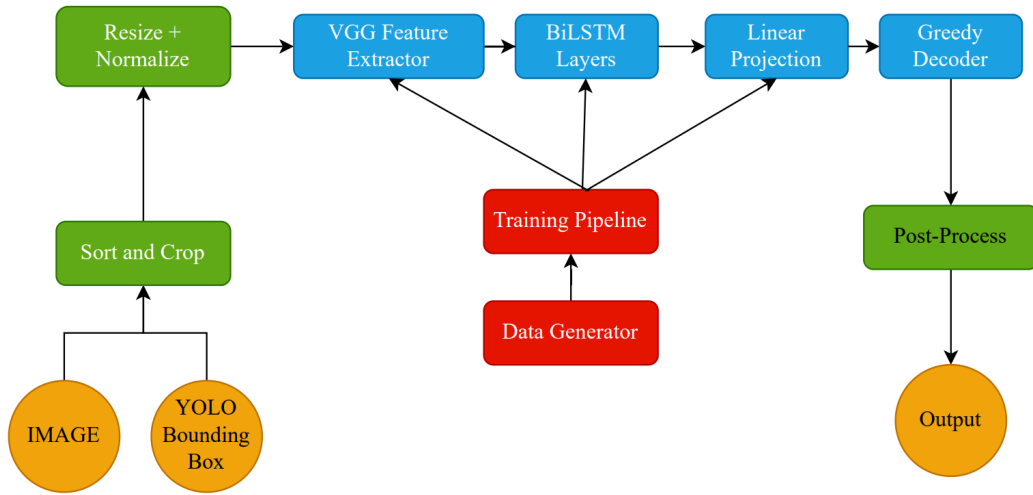


Figure 4-3 Text Detection and Recognition Pipeline

YOLOv7 outputs (bounding box information) help segregate textual information (class ‘0’ in the used dataset) which are then cropped, resized (to image height as 32) and normalized for compatibility with text recognition module. This is then passed through to a CRNN architecture composed of a Convolutional Encoder, Sequence Modeling Engine with Linear Projection, and Connectionist Temporal Classification (CTC) Decoder.

Convolutional Encoder (CNN): A CNN stack (e.g., VGG) transforms input image I from multi-dimensional $\mathbb{R}^{H \times W \times C}$ to feature maps:

$$F = f_{\text{CNN}}(I_i), \quad F \in \mathbb{R}^{H' \times W' \times C} \quad (4-7)$$

Sequence Modeling (Bidirectional LSTM): Each F becomes input to a bidirectional Long Short-Term Memory (LSTM), yielding hidden states h_t . These hidden states capture context across the text line, allowing modeling of character sequences:

$$\vec{h}_t = \text{LSTM}_f(F_t, \vec{h}_{t-1}) \quad (4-8)$$

CTC Decoder: A softmax layer projects h_t over the character vocabulary as:

$$p_{t,k} = \frac{e^{W_k h_t + b_k}}{\sum_{k'} e^{W_{k'} h_t + b_{k'}}} \quad (4-9)$$

The probability of sequence z is computed over all alignments:

$$P(z | y) = \sum_{\pi: B(\pi)=z} \prod_{t=1}^T p_{t, \pi_t} \quad (4-10)$$

The CTC loss is given by the negative logarithm of $P(z | y)$. It enables end-to-end sequence decoding without explicit character bounding boxes.

During inference, greedy decoding strategy is used which selects the most-probable character at each time step while removing blank symbols and collapsing consecutive duplicate characters. The recognized text then needs to be mapped to the appropriate circuit component whose value it represents.

4.1.4 Wire Detection

Wire detection is performed using junction points detected by the object detection model as structural anchors for circuit connectivity. All detected junctions are first transformed from normalized detection space to pixel coordinates to ensure geometric consistency within the image domain. Pairwise combinations of junction points are then analyzed to identify potential wire candidates.

For each junction pair, a displacement vector and its orientation angle are computed. Based on a predefined angular tolerance, only near horizontal and near vertical orientations are retained, while diagonal connections are rejected to conform with standard schematic drawing conventions. Valid pairs are projected onto strict axis-aligned representations to form candidate wire segments.

To improve robustness against detection noise, wire endpoints are snapped to a uniform grid and collinear segments are consolidated using proximity and overlap criteria. This produces a compact and topologically consistent set of horizontal and vertical wire segments that represent the underlying circuit wiring.

4.1.5 Terminal Point Detection

Terminal point detection focuses on identifying meaningful electrical connection points associated with wires and components. Junction detections from the object detection model serve as primary terminal candidates, representing wire endpoints, intersections, and branching locations.

Once the final wire set is established, terminal points are refined by analyzing intersections between axis-aligned wire segments and component bounding boxes. Each intersection corresponds to a valid electrical terminal where a wire connects to a component edge. Intersection conditions are evaluated analytically to ensure precise localization of entry and exit points.

The resulting terminal points form a complete connectivity interface between wires and components, enabling accurate circuit graph construction and frontend mapping. This structured representation supports subsequent tasks such as netlist generation, interactive visualization, and automated circuit understanding.

4.1.6 Integration Algorithm

From the individual outputs from each of the aforementioned modules, the task is now to generate a structured JSON representation of the circuit which will be used to generate a corresponding SPICE netlist and also contribute in the schematic generation further down the line.

The integration between all these different outputs, in a simplistic form, would consist of the following steps:

- Get a list of components and their corresponding bounding box coordinates (from YOLOv7).
- Get a list of text data and their corresponding bounding box coordinates (from OCR).
- Map the component to the corresponding component value (text) based on the minimum Euclidean distance between a particular component and texts on the circuit.
- Once the terminals are determined based on the bounding boxes, match those terminal points to the nearest nodes in the circuit.
- Store the accumulated information in a JSON file.

4.1.7 SPICE Netlist Generation

A netlist is a file that describes the connections between components in an electronic circuit. It acts as a blueprint, specifying which components are connected and how, without detailing the physical layout or specific component placement.

A SPICE netlist is a text-based representation of an electrical circuit used for simulation by a SPICE engine. It provides all the necessary information about the components, their connections, and their electrical properties, enabling analysis such as transient response, DC/AC sweep, or frequency domain behavior.

The netlist serves as input to a circuit simulation engine (NgSpice), which then performs numerical computations to evaluate how the circuit behaves under specified conditions.

For each component, the SPICE netlist contains the information about its name, the nodes it is connected to on either side and the component's parameters or model (if necessary) in the following format:

$$\langle \text{Component_Name} \rangle \ \langle \text{Node1} \rangle \ \langle \text{Node2} \rangle \ \langle \text{Value} \rangle$$

After the JSON file with abundant information about the components, their values, and the nodes they are connected to is generated, the next step is to convert said JSON file into a netlist format (a `.txt` file which follows the appropriate conventions).

4.1.8 Digital Schematic Generation

The JSON file built by integrating all the information about the hand-drawn circuit is to be manipulated through different JavaScript libraries to generate a digital, editable schematic of the circuit. As briefly discussed in the Requirements section, Chart.js is to be used to map each component in JSON to a visual block (a symbol for the component as given by the KiCad symbol library). For each component, a graphical symbol needs to be rendered at a certain position and connection lines need to be drawn between the terminals and node positions. A rough pseudocode for the same is as follows:

Algorithm 1 Frontend Circuit Rendering

Require: Circuit JSON description J

Ensure: Rendered circuit schematic

```

for all component  $c \in J.components$  do
    Draw schematic symbol of type  $c.type$  at position  $c.position$ 
    Attach text label  $c.value$  near component position
end for
for all connection  $k \in J.connections$  do
    Retrieve terminal position from  $k.from$  and terminal index
    Retrieve node position from  $k.node$ 
    Draw wire between terminal and node positions
end for
return Rendered schematic

```

In order to make the schematic editable, Chart.js allows for draggable elements using inbuilt functions like `onDrag`, transformation handles for resizing and rotating, with

added functions (and work-arounds in some cases) for integration of editing features like editing component value, delete/add components, etc. Changes can be saved by updating the JSON structure on every interaction.

4.1.9 Backend Development

The backend is to be constructed using Next.js and FastAPI, functioning as the core middleware responsible for routing, processing, and orchestrating the system’s ML pipeline and user interactions. All endpoints follow a RESTful API design, ensuring stateless and scalable communication between the frontend and the server.

Upon receiving a hand-drawn circuit image through an API endpoint like `/upload`, the image is saved to a temporary storage location. The server invokes the YOLOv7 model (via a Python subprocess or containerized inference API) to detect and localize electrical components. Simultaneously, the OCR model is invoked to extract nearby textual annotations. The result is a list of components with bounding boxes and a list of recognized texts, each with confidence scores and coordinates.

Support for user interaction with the schematic is also handled by the backend. Operations such as component deletion, repositioning, wire addition, and modification of values are processed through specific API endpoints. Each update is stored in the database, maintaining a real-time editable circuit representation. This allows the frontend to continuously reflect changes without direct manipulation of the database, ensuring security and stability of the system.

4.1.10 LLM Integration

To enhance interaction with the circuit beyond GUI manipulation, a large language model (LLM) is to be integrated, facilitated through LangChain and LangGraph. LangChain constructs the agent architecture responsible for handling user queries, such as, “Explain the purpose of this resistor.”, “Identify possible errors in this circuit.”, etc. The agent receives contextual data from the backend (such as the circuit’s JSON structure or SPICE netlist), formats the prompt, and forwards it to the LLM through a secure API. LangGraph manages the session state, context memory, and branching logic for multi-turn conversations. This enables the system to remember previously discussed components or simulation outputs across queries.

4.2 Wire Detection Methodology

The detection of wires in circuit diagrams requires a systematic approach that leverages geometric constraints inherent to electrical schematics. This section presents a comprehensive methodology for wire detection and terminal point analysis based on

junction points detected by the YOLOv7 object detection framework. The fundamental assumption underlying this work is that circuit wires are predominantly axis-aligned, meaning they run either horizontally or vertically with respect to the image coordinate system.

The image domain is defined as the discrete set $\Omega = \{(x, y) \in \mathbb{Z} : 0 \leq x < W, 0 \leq y < H\}$ where W and H represent the image width and height in pixels respectively. The YOLO detection framework operates in a normalized coordinate space $\mathcal{U} = [0, 1]^2$, necessitating a transformation to pixel coordinates for subsequent geometric analysis.

4.2.1 Coordinate Denormalization

The YOLO object detection framework produces detection tuples in normalized form. Each detection is represented as a quintuple $d = (c, x_n, y_n, w_n, h_n)$ where $c \in \mathbb{N}$ denotes the class label, $(x_n, y_n) \in [0, 1]^2$ represents the normalized bounding box center, and (w_n, h_n) specifies the normalized dimensions. For wire detection, only junction points are relevant, identified by class label $c = 1$.

The denormalization operator transforms normalized coordinates to pixel space through the affine map $\mathcal{N}^{-1} : \mathcal{U} \rightarrow \mathbb{R}^2$ defined as:

$$\mathcal{N}^{-1}(x_n, y_n) = \begin{pmatrix} W \cdot x_n \\ H \cdot y_n \end{pmatrix} \quad (4-11)$$

This transformation constitutes a diagonal linear map with matrix representation $\text{diag}(W, H)$. The operator is bijective with determinant $WH \neq 0$, admitting the inverse $\mathcal{N}(x, y) = (x/W, y/H)$. The pixel-space junction set is thus constructed as $J = \{\mathbf{j} = \mathcal{N}^{-1}(x_n^{(d)}, y_n^{(d)}) : d \in \mathcal{J}\}$ where $\mathcal{J}$ denotes the subset of detections with class label unity.

4.2.2 Pairwise Junction Analysis

The wire detection algorithm operates by examining all possible pairs of junction points. For a junction set of cardinality $n = |J|$, the number of distinct unordered pairs requiring analysis is $\binom{n}{2} = n(n-1)/2$. Each pair $(\mathbf{j}_i, \mathbf{j}_k)$ with $i < k$ generates a displacement vector:

$$\mathbf{d}^{\leftrightarrow} = \mathbf{j}_k - \mathbf{j}_i = \begin{pmatrix} \Delta x \\ \Delta y \end{pmatrix} \quad (4-12)$$

where $\Delta x = j_{k,x} - j_{i,x}$ and $\Delta y = j_{k,y} - j_{i,y}$ represent the horizontal and vertical displacements respectively. The magnitude of this displacement vector, given by $\|\mathbf{d}^{\leftrightarrow}\| = \sqrt{\Delta x^2 + \Delta y^2}$, represents the Euclidean distance between the junction pair.

4.2.3 Angular Orientation Computation

The orientation of the displacement vector is computed using the four-quadrant arctangent function, which correctly resolves the angle in all quadrants of the Cartesian plane. The signed orientation angle $\theta \in (-\pi, \pi]$ is defined as:

$$\theta = \text{atan2}(\Delta y, \Delta x) \quad (4-13)$$

The atan2 function is defined piecewise to handle all possible combinations of signs for its arguments. When $\Delta x > 0$, the angle equals $\arctan(\Delta y/\Delta x)$. When $\Delta x < 0$ and $\Delta y \geq 0$, a correction of $+\pi$ is applied. When $\Delta x < 0$ and $\Delta y < 0$, a correction of $-\pi$ is applied. The degenerate cases where $\Delta x = 0$ yield $\pm\pi/2$ depending on the sign of Δy .

For classification purposes, the angle is converted to a positive representation in degrees using the transformation $\tilde{\theta} = (180\theta/\pi) \bmod 360$, yielding values in the range $[0^\circ, 360^\circ)$.

4.2.4 Horizontal and Vertical Line Classification

The classification of junction pairs as horizontal, vertical, or diagonal is accomplished through angular tolerance zones. The tolerance parameter $\epsilon = 18^\circ$ represents 10% of the half-circle, providing sufficient margin to accommodate minor deviations from perfect alignment while maintaining classification accuracy.

Horizontal Acceptance Region: The set of angles classified as horizontal is defined as the union:

$$\Theta_H = [0^\circ, 18^\circ] \cup [162^\circ, 198^\circ] \cup [342^\circ, 360^\circ] \quad (4-14)$$

This region captures angles within $\pm 18^\circ$ of the positive x -axis (0° and 360°) and within $\pm 18^\circ$ of the negative x -axis (180°).

Vertical Acceptance Region: The set of angles classified as vertical is defined as:

$$\Theta_V = [72^\circ, 108^\circ] \cup [252^\circ, 288^\circ] \quad (4-15)$$

This region captures angles within $\pm 18^\circ$ of the positive y-axis (90°) and within $\pm 18^\circ$ of the negative y-axis (270°).

Diagonal Rejection Region: The complement of the horizontal and vertical regions constitutes the diagonal rejection region:

$$\Theta_D = [0^\circ, 360^\circ) \setminus (\Theta_H \cup \Theta_V) \quad (4-16)$$

Junction pairs whose orientation falls within Θ_D are rejected as they do not represent valid axis-aligned wires.

The classification function $O : [0^\circ, 360^\circ) \rightarrow \{H, V, \emptyset\}$ maps each angle to its corresponding orientation category. The total angular coverage of valid classifications is $(|\Theta_H| + |\Theta_V|)/360^\circ = 144^\circ/360^\circ = 40\%$, indicating that 40% of angular space is accepted as orthogonal while 60% is rejected as diagonal.

4.2.5 Orthogonal Projection and Axis Alignment

For junction pairs with valid orthogonal classification, the detected endpoints must be projected onto perfectly axis-aligned lines to ensure strict horizontal or vertical wire representation. This projection compensates for minor deviations from perfect alignment that fall within the tolerance zone.

Horizontal Wire Projection: For a junction pair $(\mathbf{j}_i, \mathbf{j}_k)$ with classification $O(\tilde{\theta}) = \text{HORIZONTAL}$, the aligned wire is constructed by computing the mean y-coordinate:

$$\bar{y} = \frac{j_{i,y} + j_{k,y}}{2} \quad (4-17)$$

The projected wire endpoints are then defined as $\mathbf{s} = (j_{i,x}, \bar{y})$ and $\mathbf{e} = (j_{k,x}, \bar{y})$. Both endpoints share the same y-coordinate, ensuring perfect horizontal alignment.

Vertical Wire Projection: For a junction pair with classification $O(\tilde{\theta}) = \text{VERTICAL}$, the aligned wire is constructed by computing the mean x-coordinate:

$$\bar{x} = \frac{j_{i,x} + j_{k,x}}{2} \quad (4-18)$$

The projected wire endpoints are $\mathbf{s} = (\bar{x}, j_{i,y})$ and $\mathbf{e} = (\bar{x}, j_{k,y})$. Both endpoints share the same x-coordinate, ensuring perfect vertical alignment.

Projection Error Bound: For a junction pair with orientation angle $\tilde{\theta} \in [0^\circ, \epsilon]$ and inter-junction distance $L = \|\mathbf{d}^{\leftrightarrow}\|$, the maximum perpendicular displacement of any endpoint from its original position is bounded by:

$$\delta = \frac{|\Delta y|}{2} \leq \frac{L \sin(\epsilon)}{2} \quad (4-19)$$

For the tolerance $\epsilon = 18^\circ$ with $\sin(18^\circ) \approx 0.309$, this yields $\delta \approx 0.155L$. Thus, for a 100-pixel wire, the maximum projection displacement is approximately 15.5 pixels.

4.2.6 Grid Quantization

To ensure coordinate consistency and facilitate subsequent merging operations, all wire endpoints are quantized to a uniform grid. The grid structure is defined by the integer lattice $\mathcal{G} = g\mathbb{Z} \times g\mathbb{Z}$ where $g \in \mathbb{N}^+$ is the grid resolution, set to $g = 16$ pixels in this implementation.

Snap Function: The scalar quantization operator $\text{snap} : \mathbb{R} \rightarrow g\mathbb{Z}$ is defined as:

$$\text{snap}(v) = g \cdot \text{round}\left(\frac{v}{g}\right) = g \cdot \left\lfloor \frac{v}{g} + \frac{1}{2} \right\rfloor \quad (4-20)$$

The vectorized extension $\text{Snap} : \mathbb{R}^2 \rightarrow \mathcal{G}$ applies the scalar operator component-wise, yielding $\text{Snap}(\mathbf{p}) = (\text{snap}(p_x), \text{snap}(p_y))$.

Quantization Error Bounds: For any scalar value $v \in \mathbb{R}$, the quantization error satisfies $|v - \text{snap}(v)| \leq g/2$. For any vector $\mathbf{p} \in \mathbb{R}^2$, the error bounds are:

$$\|\mathbf{p} - \text{Snap}(\mathbf{p})\|_\infty \leq \frac{g}{2}, \quad \|\mathbf{p} - \text{Snap}(\mathbf{p})\|_2 \leq \frac{g\sqrt{2}}{2} \quad (4-21)$$

For $g = 16$ pixels, the maximum coordinate error is 8 pixels and the maximum Euclidean error is approximately 11.3 pixels.

4.2.7 Wire Segment Consolidation

The pairwise analysis of junctions frequently generates multiple overlapping wire segments that represent the same physical wire. The consolidation algorithm merges collinear segments within a tolerance threshold to produce a minimal set of non-redundant wires.

Canonical Axis-Aligned Representation: Each wire segment is transformed to a canonical form $\sigma = (a, \ell_{\min}, \ell_{\max}, \omega)$ where a denotes the fixed axis coordinate

(y -coordinate for horizontal wires, x -coordinate for vertical wires), $[\ell_{\min}, \ell_{\max}]$ specifies the extent along the variable axis, and $\omega \in \{H, V\}$ indicates the orientation.

For a horizontal wire $w = (s, e, H)$, the canonical form is $\sigma(w) = (s_y, \min(s_x, e_x), \max(s_x, e_x), H)$. For a vertical wire, the transformation is $\sigma(w) = (s_x, \min(s_y, e_y), \max(s_y, e_y), V)$. The segment length is defined as $L(\sigma) = \ell_{\max} - \ell_{\min}$.

Mergeability Relation: Two segments σ_1 and σ_2 with identical orientation are mergeable, denoted $\sigma_1 \sim \sigma_2$, if and only if:

$$|a_1 - a_2| \leq \tau \wedge \ell_{\min}(\sigma_2) \leq \ell_{\max}(\sigma_1) + \tau \quad (4-22)$$

where $\tau = g$ is the grid tolerance and the segments are ordered such that $\ell_{\min}(\sigma_1) \leq \ell_{\min}(\sigma_2)$. The first condition ensures proximity along the fixed axis, while the second condition requires overlap or near-touching along the variable axis.

Merge Operator: For mergeable segments $\sigma_1 \sim \sigma_2$, the merge operation produces a consolidated segment $\sigma_1 \oplus \sigma_2 = (a_{\text{merged}}, \ell_{\text{merged}, \min}, \ell_{\text{merged}, \max}, \omega)$ where:

$$a_{\text{merged}} = \begin{cases} a_1 & \text{if } L(\sigma_1) > L(\sigma_2) \\ a_2 & \text{otherwise} \end{cases} \quad (4-23)$$

$$\ell_{\text{merged}, \min} = \min(\ell_{\min}(\sigma_1), \ell_{\min}(\sigma_2)), \quad \ell_{\text{merged}, \max} = \max(\ell_{\max}(\sigma_1), \ell_{\max}(\sigma_2)) \quad (4-24)$$

The length-weighted axis selection preserves the axis coordinate of the longer segment, which is assumed to be more geometrically accurate. The extent bounds are computed as the union of the individual extents.

Merge Monotonicity: The merge operation is length-non-decreasing, satisfying:

$$L(\sigma_1 \oplus \sigma_2) \geq \max(L(\sigma_1), L(\sigma_2)) \quad (4-25)$$

The proof follows from the observation that $L(\sigma_1 \oplus \sigma_2) = \max(\ell_{\max}(\sigma_1), \ell_{\max}(\sigma_2)) - \min(\ell_{\min}(\sigma_1), \ell_{\min}(\sigma_2))$. In the containment case where one segment contains the other, the merged length equals the longer segment. In the overlapping case, the merged length exceeds both individual lengths.

The consolidation algorithm proceeds by first separating segments by orientation, then sorting each group by axis coordinate and extent minimum. A linear scan merges consecutive mergeable segments, producing the final consolidated wire set.

4.3 Terminal Detection

Terminal detection refers to the identification of connection points between wire segments and circuit components. These intersection points are critical for establishing the circuit topology and enabling subsequent netlist extraction. This section presents the mathematical framework for detecting wire-component intersections based on axis-aligned bounding box geometry.

4.3.1 Component Bounding Box Representation

Circuit components detected by YOLOv7 are represented as axis-aligned bounding boxes (AABBs). For a detection tuple $d = (c, x_n, y_n, w_n, h_n)$ where $c \notin \{0, 1\}$ (excluding text and junction classes), the pixel-space bounding box is defined as:

$$\mathcal{B} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \quad (4-26)$$

The corner coordinates are computed from the normalized detection parameters as:

$$x_{\min} = \left(x_n - \frac{w_n}{2}\right) W, \quad x_{\max} = \left(x_n + \frac{w_n}{2}\right) W \quad (4-27)$$

$$y_{\min} = \left(y_n - \frac{h_n}{2}\right) H, \quad y_{\max} = \left(y_n + \frac{h_n}{2}\right) H \quad (4-28)$$

The boundary of the bounding box $\partial\mathcal{B}$ consists of four edges: the left edge at $x = x_{\min}$, the right edge at $x = x_{\max}$, the top edge at $y = y_{\min}$, and the bottom edge at $y = y_{\max}$.

4.3.2 Wire Component Intersection Analysis

Consolidated wire segments are represented in parametric form for intersection analysis. A horizontal wire with y -coordinate y_w and x -extent $[x_{\min}, x_{\max}]$ defines the line segment:

$$\mathcal{L}_H = \{(x, y_w) : x \in [x_{\min}, x_{\max}]\} \quad (4-29)$$

Similarly, a vertical wire with x -coordinate x_w and y -extent $[y_{\min}, y_{\max}]$ defines:

$$\mathcal{L}_V = \{(x_w, y) : y \in [y_{\min}, y_{\max}]\} \quad (4-30)$$

4.3.3 Horizontal Wire Intersection Conditions

The intersection of a horizontal wire with a component bounding box can occur at either the left edge or the right edge of the box. The intersection conditions are derived from the geometric constraints of axis-aligned segments.

Horizontal Wire–Left Edge Intersection: The horizontal wire $\mathcal{L}_H$ with parameters $(y_w, x_{\min}, x_{\max})$ intersects the left edge of bounding box $\mathcal{B} = [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ at point $(x_{\min}, y_w)$ if and only if:

$$\Phi_L(\mathcal{L}_H, \mathcal{B}) = (y_{\min} \leq y_w \leq y_{\max}) \wedge (x_{\min} \leq x_{\min} \leq x_{\max}) \quad (4-31)$$

The first condition $(y_{\min} \leq y_w \leq y_{\max})$ ensures vertical alignment, requiring that the wire's y -coordinate falls within the vertical extent of the box. The second condition $(x_{\min} \leq x_{\min} \leq x_{\max})$ ensures horizontal overlap, requiring that the left edge x -coordinate falls within the horizontal extent of the wire.

The proof of necessity follows from the observation that if an intersection exists at $(x_{\min}, y_w)$, then this point must simultaneously lie on the wire (requiring $y = y_w$ and $x \in [x_{\min}, x_{\max}]$) and on the left edge (requiring $x = x_{\min}$ and $y \in [y_{\min}, y_{\max}]$). Sufficiency is established by verifying that when both conditions hold, the point $(x_{\min}, y_w)$ belongs to both the wire segment and the box boundary.

Horizontal Wire–Right Edge Intersection: The horizontal wire $\mathcal{L}_H$ intersects the right edge of $\mathcal{B}$ at point $(x_{\max}, y_w)$ if and only if:

$$\Phi_R(\mathcal{L}_H, \mathcal{B}) = (y_{\min} \leq y_w \leq y_{\max}) \wedge (x_{\min} \leq x_{\max} \leq x_{\max}) \quad (4-32)$$

The structure mirrors that of the left edge intersection, with $x_{\max}$ replacing $x_{\min}$ in the horizontal overlap condition.

4.3.4 Vertical Wire Intersection Conditions

The intersection of a vertical wire with a component bounding box can occur at either the top edge or the bottom edge of the box.

Vertical Wire–Top Edge Intersection: The vertical wire $\mathcal{L}_V$ with parameters $(x_w, y_{\min}, y_{\max})$ intersects the top edge of bounding box $\mathcal{B}$ at point $(x_w, y_{\min})$ if and only if:

$$\Phi_T(\mathcal{L}_V, \mathcal{B}) = (x_{\min} \leq x_w \leq x_{\max}) \wedge (y_{\min} \leq y_{\min} \leq y_{\max}) \quad (4-33)$$

The first condition $(x_{\min} \leq x_w \leq x_{\max})$ ensures horizontal alignment, requiring that the wire's x -coordinate falls within the horizontal extent of the box. The second condition $(y_{\min} \leq y_{\min} \leq y_{\max})$ ensures vertical overlap, requiring that the top edge y -coordinate falls within the vertical extent of the wire.

Vertical Wire–Bottom Edge Intersection: The vertical wire $\mathcal{L}_V$ intersects the bottom edge of $\mathcal{B}$ at point $(x_w, y_{\max})$ if and only if:

$$\Phi_B(\mathcal{L}_V, \mathcal{B}) = (x_{\min} \leq x_w \leq x_{\max}) \wedge (y_{\min} \leq y_{\max} \leq y_{\max}) \quad (4-34)$$

4.3.5 Complete Intersection Set Construction

The intersection set for a single wire-box pair aggregates all valid intersections across the relevant edges.

Wire-Box Intersection Set: For wire w with orientation ω and bounding box $\mathcal{B}$, the intersection set is defined as:

$$\mathcal{I}(w, \mathcal{B}) = \begin{cases} \{(x_{\min}, y_w) : \Phi_L\} \cup \{(x_{\max}, y_w) : \Phi_R\} & \text{if } \omega = H \\ \{(x_w, y_{\min}) : \Phi_T\} \cup \{(x_w, y_{\max}) : \Phi_B\} & \text{if } \omega = V \end{cases} \quad (4-35)$$

Intersection Cardinality Bound: For each wire-box pair, the cardinality of the intersection set is bounded by $|\mathcal{I}(w, \mathcal{B})| \leq 2$. This bound reflects the geometric reality that a straight axis-aligned wire can enter a rectangular box through one edge and exit through the opposite edge, yielding at most two intersection points.

The global intersection set aggregates all wire-box intersections across the complete wire set $\mathcal{W}$ and component box set $\mathcal{C}$:

$$\mathcal{I}_{\text{total}} = \bigcup_{w \in \mathcal{W}} \bigcup_{\mathcal{B} \in \mathcal{C}} \mathcal{I}(w, \mathcal{B}) \quad (4-36)$$

4.3.6 Terminal Point Significance

The detected intersection points serve as terminal connection points in the circuit topology. Each intersection $(x, y) \in \mathcal{I}_{\text{total}}$ indicates a location where a wire connects to a component, enabling the construction of a circuit graph where components are nodes and wires are edges. This representation facilitates subsequent circuit analysis tasks including netlist generation, connectivity verification, and schematic reconstruction.

4.4 Line Detection via Path Finding

This work presents a pipeline for detecting lines in skeletonized circuit images by identifying endpoints and constructing a graph representation using a Multi-Head Breadth-First Search (BFS) algorithm. The methodology consists of two primary stages: endpoint detection from object detection bounding boxes, and graph construction via skeleton traversal.

4.4.1 Endpoint Detection

The endpoint detection strategy differs based on the object class detected by the upstream object detector:

Class 1 Objects (Junction Points): For junction points, a **single endpoint** is computed by calculating the bounding box center (x_c, y_c) and snapping to the nearest skeleton pixel within an expanded search region (margin = 8 pixels). The closest skeleton point is returned as the endpoint.

Class 2+ Objects (Line Segments): For line segments, **two endpoints** are computed using bounding box border intersection. Skeleton pixels that lie on or near the bounding box border (within tolerance) are identified. Among these border hits, true topological endpoints (degree = 1) are preferred. The farthest pair of points is selected as the two endpoints.

4.4.2 Graph Construction via Multi-Head BFS

The Multi-Head BFS algorithm constructs a graph where **nodes** represent detected endpoints and **edges** represent skeleton connectivity between endpoints.

Each node occupies a disk region of radius r centered at the endpoint. A `node_id_map` is precomputed where each pixel stores:

$$\text{node_id_map}[y][x] = \begin{cases} i & \text{if pixel is within radius } r \text{ of node } i \\ -1 & \text{otherwise} \end{cases}$$

For each source node i , all pixels within the source node's disk are marked as visited. The BFS queue is initialized with skeleton pixels at the disk boundary. The algorithm traverses along skeleton pixels using 8-connectivity. When encountering a pixel belonging to another node j , the edge (i, j) is recorded and that branch stops. This continues until all reachable skeleton paths are explored.

Self-Edge Removal: After constructing the adjacency graph, edges connecting endpoints that belong to the same detected object (i.e., endpoint pairs from the same bounding box) are removed. This ensures the graph represents only inter-object connections.

4.5 Use-Case Diagram

The use-case diagram in Figure 4-4 outlines the CIRCUITRON system's functionality depicting the interactions between users and the system. The primary actor is the user, who can be a student, instructor, or engineer. The system begins with the user uploading a hand-drawn circuit image.

Upon submission, CIRCUITRON automatically preprocesses the image to enhance clarity and prepares it for further analysis. The system then detects components (e.g., resistors, capacitors) using YOLOv7 and identifies wires and nodes through techniques like junction-based wire detection. Simultaneously, it extracts text labels (e.g., component values) using OCR and maps them to the corresponding components.

Once the circuit structure is parsed, the system generates an editable schematic, allowing the user to manually correct any errors before proceeding. After validation, the system converts the schematic into a SPICE-compatible netlist and runs a simulation using Ngspice/PySpice. The results, including voltage and current graphs, are displayed to the user. Additionally, the system integrates an AI assistant to answer circuit-related questions, enhancing the learning and troubleshooting experience.

The "includes" relationship is used for mandatory sub-tasks like component detection and OCR, which must always execute as part of core workflows. The "extends" relationship marks optional features like manual corrections or AI Q&A, which enhance functionality but aren't required for basic operation. The diagram effectively outlines the system's scope, emphasizing its applications (or use-cases).

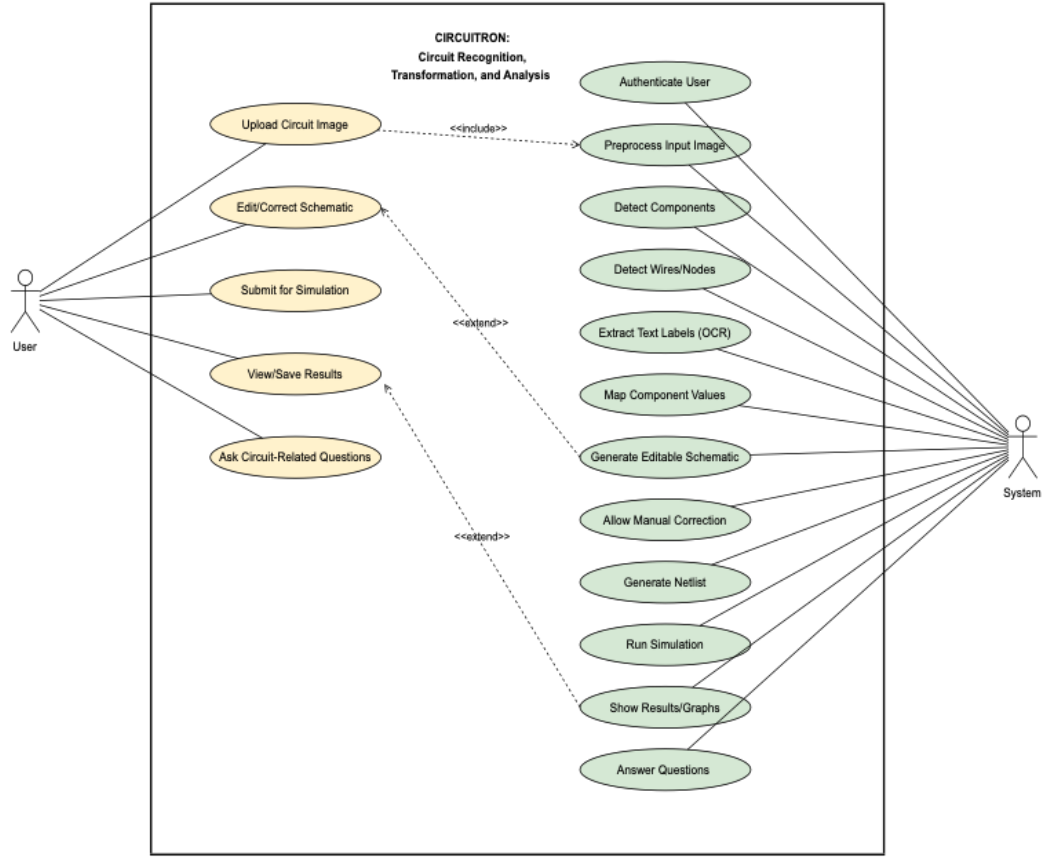


Figure 4-4 Use-Case Diagram for CIRCUITRON System

4.6 Performance Metrics Expectation

No matter how rigorously the models are trained and fine-tuned, there is always room for error due to edge cases like unclear handwriting and similar-looking components. As such, for any project, it is important to define appropriate performance metrics which indicate when the models can be considered “good-enough” for the required purpose. Table 4-1 shows the chosen performance metrics and their target values on achieving which, the training may be stopped.

Table 4-1 Expected Performance Metric Values for Individual Modules

Pipeline Stage	Performance Metric	Target Value/Expectation
Component Detection	mAP@0.5	0.90
Text Recognition	Character Accuracy	0.75

4.6.1 Component Detection Metric

mAP@0.5 (mean Average Precision at IoU threshold of 0.5) evaluates how well the model detects components with correct class labels and bounding boxes. It calculates

the Average Precision (Average Precision (AP)) for each class and then takes the mean across all classes. An IoU (Intersection over Union) threshold of 0.5 means a predicted bounding box is considered a correct match if it overlaps with the ground truth box by at least 50%.

$$\text{IoU} = \frac{\text{Area of Overlap}}{\text{Area of Union}} \quad (4-37)$$

A high mAP@0.5 means detections are both precise and complete, minimizing missed or misclassified components.

4.6.2 Text Recognition Metric

Character Accuracy measures how many characters in the predicted text match the actual characters:

$$\text{Character Accuracy} = \frac{\text{Number of Correct Characters}}{\text{Total Number of Characters in Ground Truth}} \quad (4-38)$$

In circuit diagrams, values like $10k$, 100Ω , $1\mu F$ are crucial. OCR must correctly read these values, since even a small error (e.g., mistaking ‘k’ as ‘1’) leads to incorrect simulation behavior. A high character accuracy ensures value-to-component mapping is reliable.

4.6.3 Terminal and Node Detection Metrics

Precision measures how many detected terminals (or nodes) are correct whereas recall measures how many actual terminals (or nodes) were correctly detected:

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}} \quad (4-39)$$

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}} \quad (4-40)$$

Terminals define where components connect to wires, which influences circuit structure. High precision ensures the system doesn’t falsely identify noise as terminals, avoiding wrong wire mappings. High recall ensures the system doesn’t miss valid terminal points, which would cause incomplete circuits. Likewise, missing nodes can create open circuits, and false nodes can misrepresent actual connectivity. So high values of both precision and recall are crucial for both terminal and node detection.

4.7 Error Handling

In any project with a number of individual modules working together, the overall error not only comes from said individual modules but also from their integration due to propagating errors from one module to another. To handle errors in such a case, foremost the errors in the output of the individual modules need to be controlled and only then can the accumulated error be kept in check. The performance metrics tabulated in the previous section are to be strictly adhered to if there's any hope of generating the SPICE netlist (and digital schematic) with nominal errors.

For the reduction of false matches and OCR noise, post-processing techniques such as value format validation, confidence filtering, and text cleanup can be implemented. Value format validation ensures only strings that match known (and appropriate) electrical units, say $10k$, $1\mu F$, $5V$ are retained. Confidence filtering helps discard OCR results with low confidence scores, and text cleanup normalizes ambiguous symbols such as converting “O” to “0”, “ μ ” to “u”.

A basic pseudocode for sample value format validation is as follows:

Algorithm 2 Validation and Filtering of OCR Component Values

Require: OCR detected text entries T

Ensure: Filtered list of valid component values V

Define regular expression patterns for resistance, capacitance, and voltage

$V \leftarrow \emptyset$

for all text entry $t \in T$ **do**

if t .text matches any defined pattern **then**

 Append t to V

end if

end for

return V

The list of components shown here is in no way meant to be exhaustive; rather it is to show how this pseudocode ensures that only valid electrical component values are used in the final component-to-value mapping and schematic generation.

To prevent propagating error (error from the output of YOLOv7 bleeding into OCR, for instance), a combined confidence threshold checking mechanism can be implemented which marks an item as uncertain if the confidence score from YOLOv7 is less than a certain value (e.g., 0.6) and the same from OCR is less than a certain value (e.g., 0.85). For more robustness, a human-in-the-loop mechanism can be implemented which prompts the user to manually approve or correct OCR-predicted values, fix connection errors, etc. This can be implemented to trigger when the confidence score is below the

set threshold or when other error handling mechanisms such as value format validation fail. The pseudocode for this is as follows:

Algorithm 3 Confidence-Based Error Checking and Validation

Require: Module outputs O , confidence threshold ω , format validation function f

Ensure: Valid outputs V and uncertain outputs U

```

 $V \leftarrow \emptyset$ 
 $U \leftarrow \emptyset$ 
for all item  $o \in O$  do
    if  $o.\text{confidence} \geq \omega \wedge f(o.\text{value})$  then
        Append  $o$  to  $V$ 
    else
        Append  $o$  to  $U$ 
    end if
end for
if  $U \neq \emptyset$  then
    Trigger manual override
end if
return ( $V, U$ )

```

4.7.1 Image Augmentation

Since the dataset contains handwritten circuit diagrams captured under variable lighting and orientations, several augmentation techniques were applied to improve model generalization including Contrast Enhancement (global) and Adaptive Brightness Adjustment by factors of 0.8 and 1.2. Furthermore, geometric augmentation techniques, like spatial flipping (both horizontally and vertically) and rotation (e.g., 90° or random rotation) proved to be unwise operations for this project. For example, if a “C” is rotated by 90°, the “C” now resembles a “U”, and the same applies to flipping (“M” might look like “W”, for example). Rather than increasing accuracy, it was realized that using rotation or flipping would simply confuse the model. However, continuous testing with other forms of augmentation will continue in hopes of improving the model.

4.7.2 Fine-tuning YOLOv5 and YOLOv7 for Component Detection

YOLOv5 was initially used for benchmarking, and later upgraded to YOLOv7 for better accuracy and inference speed (comparison shown in the Results section). Training and fine-tuning were done using the CGHD dataset after converting into YOLO-compatible format.

The training parameters employed are as follows:

- **Number of classes:** 61
- **Training split:** 70% training, 10% validation, 20% testing

- **Epochs:** 200 (for YOLOv5), 100 (for YOLOv7)
- **Image size:** 416×416 pixels
- **Batch size:** 16 (for YOLOv5), 32 (for YOLOv7)
- **Optimizer:** Stochastic Gradient Descent (SGD) (Stochastic Gradient Descent)
- **Learning rate:** 0.01 (initial)
- **Augmentations used in training:** Mosaic Augmentation, HSV Augmentation, Random rotating/flipping

4.7.3 Implementation of Text Recognition (OCR)

To extract textual information such as resistor values, capacitor labels, and other component annotations from the circuit diagrams, a CRNN-based custom OCR model was chosen, trained and implemented. The input image in cropped form (cropped for each text component) was passed through this model after training, which returned the content of each detected text element. The position of said text element, however, was found using bounding boxes from YOLOv7. This allowed us to localize and read numerical values or labels directly from the circuit.

Three different modules, each for feature extraction, sequence modeling, and prediction were trained based on the DTRB framework. Initially, simply fine-tuning OCR models like TrOCR and EasyOCR were also attempted; however, these didn't yield usable results, ultimately requiring training from scratch. The training was done for 29 epochs.

4.7.4 Line Detection Algorithm Implementation

The line detection mechanism was engineered to exploit the semantic constraints of circuit diagrams, specifically that wires are predominantly axis-aligned (horizontal or vertical) and connect distinct components. Unlike the Probabilistic Hough Transform (PHT) which operates purely on pixel gradients, this approach uses the junction points detected by YOLOv7 as anchor nodes to reconstruct the wiring layout geometrically.

First, a pairwise analysis was performed on all junction points identified by the object detection model. The algorithm iterates through every unique pair of junctions to calculate the angle of the vector connecting them. Since hand-drawn circuits contain imperfections, an angular tolerance of $\pm 18^\circ$ was applied to classify connections as either horizontal or vertical. To ensure coordinate consistency across the diagram, the resulting segment coordinates were "snapped" to a grid using a 16-pixel quantization.

A common issue with the pairwise approach is the generation of redundant, overlapping segments for the same wire. For instance, if three junctions lie on a single line, the

Algorithm 4 Wire Classification and Alignment

Require: Set of junctions J , angular tolerance $\omega_{\text{tol}} = 18^\circ$, grid size $g = 16$

Ensure: Set of raw wire segments S

```
 $S \leftarrow \emptyset$ 
for all pairs  $(j_a, j_b) \in \text{Combinations}(J, 2)$  do
     $\Delta x \leftarrow j_b.x - j_a.x$ 
     $\Delta y \leftarrow j_b.y - j_a.y$ 
     $\theta \leftarrow \text{atan2}(\Delta y, \Delta x)$ 
    if  $|\theta| \leq \omega_{\text{tol}} \vee |\theta - 180^\circ| \leq \omega_{\text{tol}}$  then
        Horizontal alignment check
         $y_{\text{avg}} \leftarrow \frac{j_a.y + j_b.y}{2}$ 
         $y_{\text{snap}} \leftarrow \text{Round}\left(\frac{y_{\text{avg}}}{g}\right) \times g$ 
         $S \leftarrow S \cup \{(H, y_{\text{snap}}, \min(j_a.x, j_b.x), \max(j_a.x, j_b.x))\}$ 
    else if  $|\theta - 90^\circ| \leq \omega_{\text{tol}} \vee |\theta - 270^\circ| \leq \omega_{\text{tol}}$  then
        Vertical alignment check
         $x_{\text{avg}} \leftarrow \frac{j_a.x + j_b.x}{2}$ 
         $x_{\text{snap}} \leftarrow \text{Round}\left(\frac{x_{\text{avg}}}{g}\right) \times g$ 
         $S \leftarrow S \cup \{(V, x_{\text{snap}}, \min(j_a.y, j_b.y), \max(j_a.y, j_b.y))\}$ 
    end if
end for
return  $S$ 
```

algorithm produces three separate segments (A-B, B-C, and A-C). To combat this, a collinear segment merging algorithm was implemented.

The segments were first separated by orientation (horizontal/vertical) and sorted by their primary axis. The merging logic then consolidated overlapping or adjacent segments. Crucially, a length-weighted heuristic was used: when merging, the axis coordinate of the longer segment was preserved, ensuring that larger, more reliable strokes dictated the final geometric alignment.

The wire–component intersection detection algorithm identifies precise connection points between detected wire segments and component bounding boxes. Component regions are first extracted from YOLOv7 annotations and converted into pixel coordinates. Each horizontal and vertical wire is then tested for geometric intersection with the edges of every component box. The resulting intersection points are used to accurately map circuit connectivity and are optionally visualized for verification in the frontend.

While detecting terminals, a method to ensure faulty values are not accepted is to count the terminals. Unless a few three-terminal components are used, the number of terminals is likely to be even, so a mismatch in this could potentially indicate an error. In such cases, the detection can be re-run and/or the user can be notified. In case of failure to detect or associate a value, and if other error-handling mechanisms are not triggered, a

Algorithm 5 Collinear Segment Merging

Require: List of segments L , gap tolerance ω

Ensure: List of merged segments M

Sort L by primary axis, then by starting coordinate

$M \leftarrow \emptyset$

$\text{curr} \leftarrow L[0]$

for all $\text{next} \in L[1 \dots N]$ **do**

if $|\text{curr.axis} - \text{next.axis}| \leq \omega \wedge \text{next.start} \leq \text{curr.end} + \omega$ **then**

$\text{curr.start} \leftarrow \min(\text{curr.start}, \text{next.start})$

$\text{curr.end} \leftarrow \max(\text{curr.end}, \text{next.end})$

if $\text{Length}(\text{next}) > \text{Length}(\text{curr})$ **then**

Preserve axis of the longer segment

$\text{curr.axis} \leftarrow \text{next.axis}$

end if

else

$M \leftarrow M \cup \{\text{curr}\}$

$\text{curr} \leftarrow \text{next}$

end if

end for

$M \leftarrow M \cup \{\text{curr}\}$

return M

Algorithm 6 Wire–Component Intersection Detection

Require: Wire segments W , YOLO label file L , image width I_w , image height I_h

Ensure: Set of intersection points P

Parse label file L

Convert normalized bounding boxes to pixel coordinates

Discard non-component classes

Store component boxes B

Detect intersections between wires and components

$P \leftarrow \emptyset$

for all wire $w \in W$ **do**

for all box $b \in B$ **do**

if w is horizontal and crosses left or right edge of b **then**

 Add intersection point to P

else if w is vertical and crosses top or bottom edge of b **then**

 Add intersection point to P

end if

end for

end for

Draw wires, component boxes, and intersection points on image

Save output visualization

return P

placeholder can be used (e.g., $R1 = \text{UNKNOWN}$), and the system allows manual user correction before simulation as a last resort.

4.8 System Flowchart

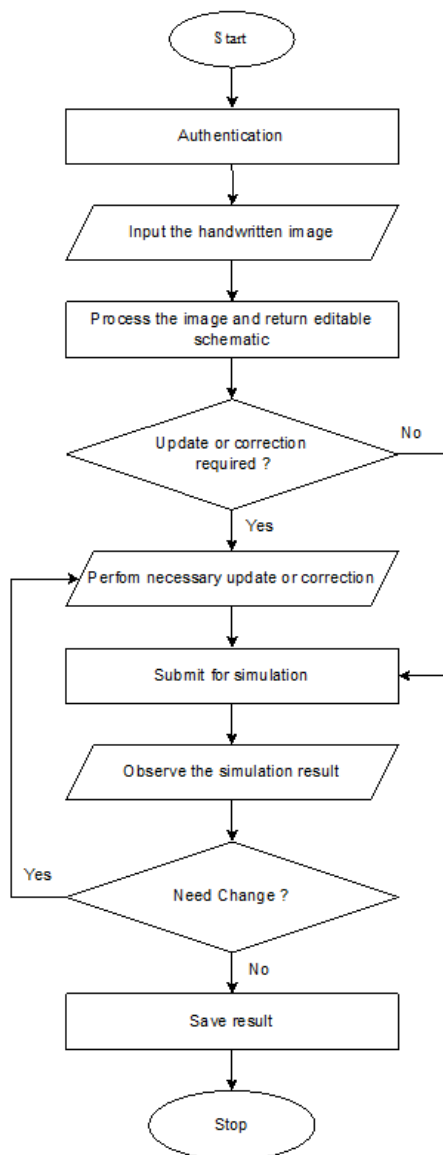


Figure 4-5 System Flowchart

The key architectural modules involved in the working of the system have been demonstrated by the block diagram in Figure 4-1. Now, the flowchart in Figure 4-5 demonstrates the motions a user needs to go through to use the final product. First, the user needs to be authenticated (optional). Once the image of a hand-drawn circuit is

uploaded, the system processes the image (internally using the techniques explained in the previous sections) and returns an editable schematic. Any necessary update or correction may be performed if necessary and then the schematic can be submitted for simulation and analysis. Relevant graphs are returned after analysis and the result can be saved for later access.

4.9 Dataset Analysis

In order to develop the system using the system for automated circuit diagram understanding, the CGHD (Circuit Graph Handwritten Diagrams) dataset is used. This dataset provides a comprehensive suite of ground truth annotations tailored to facilitate key computer vision tasks such as object detection, instance segmentation, and text recognition. While additional datasets (such as circuitVQA, Digitize-HCD datasets) may be used to fine-tune the models in the future to achieve even higher values for performance metrics, as of yet, only the CGHD dataset has been used.

4.9.1 Dataset Overview

The CGHD dataset comprises 4.99 GB of annotated data spanning over 3,000 raw images. The data is separated into folders named drafters, going from -1 through 31, and each drafter folder contains hand-sketches of 12 unique circuit designs, with each circuit drawn twice and captured under four different conditions (varying angles, lighting, and minor distortions), resulting in up to 96 images per drafter. As is convention, the dataset is to be split into (70-20-10)% of the entire data for training, validation and testing respectively.

Table 4-2 Summary of Data in the CGHD Dataset

Items	Number
Distinct Circuits	372
Raw Images	3,269
Bounding Box Annotations	248,020
Text String Annotations	85,417 (93.74% completeness)
Character Types	98
Object Classes	61

Every image is rigorously annotated and the annotations provided per image include:

- **Bounding Boxes (PASCAL VOC format)** for 61 electrical component classes, including resistors, capacitors, transistors, operational amplifiers, and textual elements.
- **Text Labels** capturing component values and identifiers (e.g., “ $10k\Omega$ ”, “C1”, “V”).

- **Instance Segmentation Polygons** (LabelMe JSON format) for pixel-level object outlines.
- **Binary Segmentation Maps** distinguishing foreground circuit strokes from background artifacts.
- **Partial Netlist Files** (in `.asc` format) for selected samples, useful for validating circuit reconstruction.

The text annotation coverage is extensive, with labels ranging from single characters (like “*R*” or “*R1*”) to full descriptors (like “Resistor”, “Variable Resistor”, etc.).

4.9.2 Class-wise Data Statistics

Table 4-3 Frequency of Each Class with its Code in the CGHD Dataset

Code	Class Name	Count	Code	Class Name	Count
1	text	91,122	32	opamp.schmitt	16
2	junction	79,202	33	optocoupler	162
3	crossover	10,180	34	ic	1,506
4	terminal	11,253	35	ic.ne555	262
5	gnd	5,501	36	ic.v_regulator	362
6	vss	1,354	37	xor	162
7	voltage.dc	335	38	and	860
8	voltage.ac	292	39	or	440
9	voltage.battery	765	40	not	496
10	resistor	14,400	41	nand	228
11	resistor.adj	1,052	42	nor	59
12	resistor.photo	65	43	probe	184
13	cap.unpol	5,841	44	probe.curr	72
14	cap.pol	2,227	45	probe.volt	32
15	cap.adj	88	46	switch	2,184
16	inductor	1,976	47	relay	89
17	inductor.ferrite	120	48	socket	184
18	inductor.coupled	10	49	fuse	226
19	transformer	483	50	speaker	410
20	diode	3,890	51	motor	108
21	diode.led	1,212	52	lamp	301
22	diode.thyrector	16	53	microphone	72
23	diode.zener	609	54	antenna	24
24	diac	40	55	crystal	112
25	triac	98	56	mechanical	32
26	thyristor	415	57	magnetic	364
27	varistor	184	58	optical	127
28	transistor.bjt	3,678	59	block	0
29	transistor.fet	977	60	explanatory	610
30	transistor.photo	177	61	unknown	32
31	opamp	752			

5 IMPLEMENTATION DETAILS

The implementation of the system to convert hand-drawn circuit diagrams into simulation-ready netlists and digital schematics follows a modular approach, aligning with the pipeline described in the architecture and methodology. Some of the methodologies explained in the previous section, that have been implemented are explained in this section.

5.1 Dataset Manipulation and Preprocessing

5.1.1 Conversion of Annotation Formats

To accommodate the multiple models in the pipeline (YOLOv7 for component detection and OCR for value extraction), annotation files provided in the CGHD dataset in Pascal VOC (XML) format were converted into compatible formats. Likewise, for EasyOCR-inspired custom-trained OCR, all text-sections needed to be cropped out from the image with corresponding labels (ground truth) and then fed to the fine-tuned model.

In Pascal VOC format, the information regarding the circuit and its components are stored in the following format as shown by the example below:

```
<annotation>
  <folder>images</folder>
  <filename>C1_D1_P2.jpg</filename>
  <path>./drafter_1/images/C1_D1_P2.jpg</path>
  <source>
    <database>CGHD</database>
  </source>
  <size>
    <width>3024</width>
    <height>4032</height>
    <depth>3</depth>
  </size>
  <segmented>0</segmented>
  <object>
    <name>voltage.dc</name>
    <pose>Unspecified</pose>
    <truncated>0</truncated>
    <difficult>0</difficult>
    <bndbox>
      <xmin>327</xmin>
      <ymin>546</ymin>
```

```

        <xmax>604</xmax>
        <ymax>838</ymax>
        <rotation>10</rotation>
    </bndbox>
</object>
</annotation>

```

For YOLO-based detection, XML annotations were converted to YOLO format (.txt) using a custom script. Each .txt file contains the class index and normalized bounding box coordinates in the format:

```
<class_id> <x_center> <y_center> <width> <height>
```

The pseudocode for the VOC to YOLO annotation conversion is as follows:

Algorithm 7 VOC to YOLO Annotation Conversion

Require: XML annotation file X , class map C , image width W , image height H

Ensure: List of YOLO formatted annotation lines Y

```

    Parse XML file  $X$  into tree structure
     $Y \leftarrow \emptyset$ 
    for all object nodes  $o$  in XML root do
        clsname  $\leftarrow$  class name of  $o$ 
        if clsname  $\notin C$  then
            continue
        end if
        clsid  $\leftarrow C[\text{clsname}]$ 
        Extract bounding box  $(x_{\min}, y_{\min}, x_{\max}, y_{\max})$  from  $o$ 
        Convert VOC coordinates to normalized YOLO format
         $x_{\text{center}} \leftarrow \frac{x_{\min} + x_{\max}}{2W}$ 
         $y_{\text{center}} \leftarrow \frac{y_{\min} + y_{\max}}{2H}$ 
         $w \leftarrow \frac{x_{\max} - x_{\min}}{W}$ 
         $h \leftarrow \frac{y_{\max} - y_{\min}}{H}$ 
        Append formatted string  $(x_{\text{center}}, y_{\text{center}}, w, h)$  to  $Y$ 
    end for
    return  $Y$ 

```

The calculations run for this conversion is as follows:

$$x_{\text{center}} = \frac{327 + 604}{2 \times 3024} = \frac{465.5}{3024} = 0.1539 \quad (5-1)$$

$$y_{\text{center}} = \frac{546 + 838}{2 \times 4032} = \frac{692}{4032} = 0.1716 \quad (5-2)$$

$$w = \frac{604 - 327}{3024} = \frac{277}{3024} = 0.0916 \quad (5-3)$$

$$h = \frac{838 - 546}{4032} = \frac{292}{4032} = 0.0724 \quad (5-4)$$

This gives the information required by YOLO in the appropriate format. It is convention to save this new annotation file in the same directory as the image.

Likewise, for OCR compatibility, the same Pascal VOC annotations were converted to Lightning Memory-Mapped Database (LMDB) format. LMDB is a high-speed key-value store where each key corresponds to an index and each value contains an image-text pair (serialized).

The pseudocode for the conversion of Pascal VOC format to OCR-compatible LMDB file format is as follows:

Algorithm 8 Image Cropping and LMDB Dataset Creation

Require: Input image file I , crop coordinates $(y_{\min}, y_{\max}, x_{\min}, x_{\max})$, target size (W, H)

Ensure: LMDB database containing image-label pairs

```

Load image  $I$  into memory
Crop image using coordinates  $[y_{\min} : y_{\max}, x_{\min} : x_{\max}]$ 
Resize cropped image to size  $(W, H)$ 
Convert cropped image to byte representation
Convert image array to PIL image
Initialize in-memory byte buffer
Save image to buffer in JPEG format
Extract byte stream from buffer
Store image and label in LMDB
Open LMDB environment with predefined map size
Begin write transaction
Store image bytes with key image-00001
Store corresponding label with key label-00001
Store total sample count with key num-samples
Commit transaction
return LMDB environment

```

After this script is run in its entirety, a file in LMDB format, as is expected by OCR, is received as shown in the example below:

```

Key: b'image-00001'
Value: b'\xff\xd8\xff\xe0\x00\x10JFIF...'

Key: b'label-00001'
Value: b'voltage.dc'

```

Key: b'num-samples'

Value: b'000001'

After necessary conversion, the dataset was ready for being plugged into the required models for fine-tuning them. But first, some experimentation was done with different forms of image augmentation techniques.

5.1.2 Image Augmentation

Since the dataset contains handwritten circuit diagrams captured under variable lighting and orientations, several augmentation techniques were applied to improve model generalization including Contrast Enhancement (global) and Adaptive Brightness Adjustment by factors of 0.8 and 1.2. Furthermore, geometric augmentation techniques, like spatial flipping (both horizontally and vertically) and rotation (e.g., 90° or random rotation) proved to be unwise operations for this project. For example, if a “C” is rotated by 90°, the “C” now resembles a “U”, and the same applies to flipping (“M” might look like “W”, for example). Rather than increasing accuracy, it was realized that using rotation or flipping would simply confuse the model. However, continuous testing with other forms of augmentation will continue in hopes of improving the model.

5.1.3 Fine-tuning YOLOv5 and YOLOv7 for Component Detection

YOLOv5 was initially used for benchmarking, and later upgraded to YOLOv7 for better accuracy and inference speed (comparison shown in the Results section). Training and fine-tuning were done using the CGHD dataset after converting into YOLO-compatible format.

The training parameters employed are as follows:

- **Number of classes:** 61
- **Training split:** 70% training, 10% validation, 20% testing
- **Epochs:** 200 (for YOLOv5), 100 (for YOLOv7)
- **Image size:** 416 × 416 pixels
- **Batch size:** 16 (for YOLOv5), 32 (for YOLOv7)
- **Optimizer:** SGD (Stochastic Gradient Descent)
- **Learning rate:** 0.01 (initial)
- **Augmentations used in training:** Mosaic Augmentation, HSV Augmentation, Random rotating/flipping

5.1.4 Implementation of Text Recognition (OCR)

To extract textual information such as resistor values, capacitor labels, and other component annotations from the circuit diagrams, a CRNN-based custom OCR model was chosen, trained and implemented. The input image in cropped form (cropped for each text component) was passed through this model after training, which returned the content of each detected text element. The position of said text element, however, was found using bounding boxes from YOLOv7. This allowed us to localize and read numerical values or labels directly from the circuit.

Three different modules, each for feature extraction, sequence modeling, and prediction were trained based on the DTRB framework. Initially, simply fine-tuning OCR models like TrOCR and EasyOCR were also attempted; however, these didn't yield usable results, ultimately requiring training from scratch. The training was done for 29 epochs.

5.1.5 Line Detection Algorithm Implementation

The line detection mechanism was engineered to exploit the semantic constraints of circuit diagrams, specifically that wires are predominantly axis-aligned (horizontal or vertical) and connect distinct components. Unlike the Probabilistic Hough Transform (PHT) which operates purely on pixel gradients, this approach uses the junction points detected by YOLOv7 as anchor nodes to reconstruct the wiring layout geometrically.

First, a pairwise analysis was performed on all junction points identified by the object detection model. The algorithm iterates through every unique pair of junctions to calculate the angle of the vector connecting them. Since hand-drawn circuits contain imperfections, an angular tolerance of $\pm 18^\circ$ was applied to classify connections as either horizontal or vertical. To ensure coordinate consistency across the diagram, the resulting segment coordinates were "snapped" to a grid using a 16-pixel quantization.

A common issue with the pairwise approach is the generation of redundant, overlapping segments for the same wire. For instance, if three junctions lie on a single line, the algorithm produces three separate segments (A-B, B-C, and A-C). To combat this, a collinear segment merging algorithm was implemented.

The segments were first separated by orientation (horizontal/vertical) and sorted by their primary axis. The merging logic then consolidated overlapping or adjacent segments. Crucially, a length-weighted heuristic was used: when merging, the axis coordinate of the longer segment was preserved, ensuring that larger, more reliable strokes dictated the final geometric alignment.

Algorithm 9 Wire Classification and Alignment

Require: Set of junctions J , angular tolerance $\omega_{\text{tol}} = 18^\circ$, grid size $g = 16$

Ensure: Set of raw wire segments S

```
 $S \leftarrow \emptyset$ 
for all pairs  $(j_a, j_b) \in \text{Combinations}(J, 2)$  do
   $\Delta x \leftarrow j_b.x - j_a.x$ 
   $\Delta y \leftarrow j_b.y - j_a.y$ 
   $\theta \leftarrow \text{atan2}(\Delta y, \Delta x)$ 
  if  $|\theta| \leq \omega_{\text{tol}} \vee |\theta - 180^\circ| \leq \omega_{\text{tol}}$  then
    Horizontal alignment check
     $y_{\text{avg}} \leftarrow \frac{j_a.y + j_b.y}{2}$ 
     $y_{\text{snap}} \leftarrow \text{Round}\left(\frac{y_{\text{avg}}}{g}\right) \times g$ 
     $S \leftarrow S \cup \{(H, y_{\text{snap}}, \min(j_a.x, j_b.x), \max(j_a.x, j_b.x))\}$ 
  else if  $|\theta - 90^\circ| \leq \omega_{\text{tol}} \vee |\theta - 270^\circ| \leq \omega_{\text{tol}}$  then
    Vertical alignment check
     $x_{\text{avg}} \leftarrow \frac{j_a.x + j_b.x}{2}$ 
     $x_{\text{snap}} \leftarrow \text{Round}\left(\frac{x_{\text{avg}}}{g}\right) \times g$ 
     $S \leftarrow S \cup \{(V, x_{\text{snap}}, \min(j_a.y, j_b.y), \max(j_a.y, j_b.y))\}$ 
  end if
end for
return  $S$ 
```

Algorithm 10 Collinear Segment Merging

Require: List of segments L , gap tolerance ω

Ensure: List of merged segments M

```
Sort  $L$  by primary axis, then by starting coordinate
 $M \leftarrow \emptyset$ 
curr  $\leftarrow L[0]$ 
for all next  $\in L[1 \dots N]$  do
  if  $|\text{curr.axis} - \text{next.axis}| \leq \omega \wedge \text{next.start} \leq \text{curr.end} + \omega$  then
    curr.start  $\leftarrow \min(\text{curr.start}, \text{next.start})$ 
    curr.end  $\leftarrow \max(\text{curr.end}, \text{next.end})$ 
    if  $\text{Length}(\text{next}) > \text{Length}(\text{curr})$  then
      Preserve axis of the longer segment
      curr.axis  $\leftarrow \text{next.axis}$ 
    end if
  else
     $M \leftarrow M \cup \{\text{curr}\}$ 
    curr  $\leftarrow \text{next}$ 
  end if
end for
 $M \leftarrow M \cup \{\text{curr}\}$ 
return  $M$ 
```

The wire–component intersection detection algorithm identifies precise connection points between detected wire segments and component bounding boxes. Component regions are first extracted from YOLOv7 annotations and converted into pixel coordinates. Each horizontal and vertical wire is then tested for geometric intersection with the edges of every component box. The resulting intersection points are used to accurately map circuit connectivity and are optionally visualized for verification in the frontend.

Algorithm 11 Wire–Component Intersection Detection

Require: Wire segments W , YOLO label file L , image width I_w , image height I_h

Ensure: Set of intersection points P

Parse label file L

Convert normalized bounding boxes to pixel coordinates

Discard non-component classes

Store component boxes B

Detect intersections between wires and components

$P \leftarrow \emptyset$

for all wire $w \in W$ **do**

for all box $b \in B$ **do**

if w is horizontal and crosses left or right edge of b **then**

 Add intersection point to P

else if w is vertical and crosses top or bottom edge of b **then**

 Add intersection point to P

end if

end for

end for

Draw wires, component boxes, and intersection points on image

Save output visualization

return P

5.2 Line Detection via Path Finding

This section presents the algorithmic implementation of the line detection pipeline. The approach involves three key procedures: constructing a spatial mapping of node regions, performing multi-head BFS traversal to discover connectivity, and filtering out spurious self-edges.

The first step in the pipeline is building a node identification map. This data structure enables constant-time lookup to determine whether a given pixel belongs to a node region. For each detected endpoint, a circular disk of radius r is marked in the map, associating all pixels within that disk with the corresponding node index. Algorithm 12 formalizes this construction.

Algorithm 12 Build Node ID Map

Require: Skeleton image S of size $H \times W$, list of nodes $\mathcal{N} = \{(x_0, y_0), \dots, (x_{n-1}, y_{n-1})\}$, radius r

Ensure: Node ID map M of size $H \times W$

```
1:  $M \leftarrow$  array of size  $H \times W$  initialized to  $-1$ 
2:  $\mathcal{D} \leftarrow \text{DiskMask}(r)$ 
3: for  $i \leftarrow 0$  to  $n - 1$  do
4:    $(n_x, n_y) \leftarrow \mathcal{N}[i]$ 
5:   for each  $(dy, dx) \in \mathcal{D}$  do
6:      $(p_y, p_x) \leftarrow (n_y + dy, n_x + dx)$ 
7:     if  $0 \leq p_y < H$  and  $0 \leq p_x < W$  then
8:        $M[p_y][p_x] \leftarrow i$ 
9:     end if
10:  end for
11: end for
12: return  $M$ 
```

Once the node ID map is constructed, the algorithm performs a breadth-first search from each node to discover its neighbors along the skeleton. The BFS starts from the boundary of each node's disk region and traverses skeleton pixels using 8-connectivity. When the traversal encounters a pixel belonging to a different node, that node is recorded as a neighbor and the branch terminates. This ensures that each skeleton path is followed until it reaches another endpoint. Algorithm 13 describes this procedure.

Algorithm 13 BFS Neighbors for Node

Require: Node index i , nodes $\mathcal{N}$, skeleton S , node ID map M , radius r

Ensure: Set of neighbor node indices $\mathcal{A}_i$

```
1:  $(n_x, n_y) \leftarrow \mathcal{N}[i]$ 
2:  $\mathcal{A}_i \leftarrow \emptyset$ 
3:  $V \leftarrow$  boolean array of size  $H \times W$  initialized to false
4:  $\mathcal{D} \leftarrow \text{DiskMask}(r)$ 
5: for each  $(dy, dx) \in \mathcal{D}$  do
6:    $(p_y, p_x) \leftarrow (n_y + dy, n_x + dx)$ 
7:   if  $0 \leq p_y < H$  and  $0 \leq p_x < W$  then
8:      $V[p_y][p_x] \leftarrow \text{true}$ 
9:   end if
10: end for
11:  $Q \leftarrow$  empty queue
12: for each  $(dy, dx) \in \mathcal{D}$  do
13:    $(p_y, p_x) \leftarrow (n_y + dy, n_x + dx)$ 
14:   for each  $(n'_y, n'_x) \in \text{Neighbors8}(p_y, p_x)$  do
15:     if  $\neg V[n'_y][n'_x]$  and  $S[n'_y][n'_x] > 0$  then
16:        $V[n'_y][n'_x] \leftarrow \text{true}$ 
17:        $Q.\text{enqueue}((n'_y, n'_x))$ 
18:     end if
19:   end for
20: end for
21: while  $Q$  is not empty do
22:    $(c_y, c_x) \leftarrow Q.\text{dequeue}()$ 
23:    $j \leftarrow M[c_y][c_x]$ 
24:   if  $j \neq -1$  and  $j \neq i$  then
25:      $\mathcal{A}_i \leftarrow \mathcal{A}_i \cup \{j\}$ 
26:     continue
27:   end if
28:   for each  $(n'_y, n'_x) \in \text{Neighbors8}(c_y, c_x)$  do
29:     if  $\neg V[n'_y][n'_x]$  and  $S[n'_y][n'_x] > 0$  then
30:        $V[n'_y][n'_x] \leftarrow \text{true}$ 
31:        $Q.\text{enqueue}((n'_y, n'_x))$ 
32:     end if
33:   end for
34: end while
35: return  $\mathcal{A}_i$ 
```

The final step removes spurious connections between endpoints that belong to the same detected object. Since line segment bounding boxes produce two endpoints each, the BFS may incorrectly connect these as neighbors via the skeleton segment within the bounding box. By maintaining a list of endpoint pairs from the same object, these self-edges are explicitly removed from the adjacency graph. Algorithm 14 implements this filtering.

Algorithm 14 Remove Self-Edges (Same-Object Connections)

Require: Adjacency $\mathcal{G}$, endpoint pairs $\mathcal{P} = \{(p_1^{(k)}, p_2^{(k)})\}_{k=1}^m$, nodes $\mathcal{N}$

Ensure: Filtered adjacency $\mathcal{G}'$

```
1:  $\mathcal{G}' \leftarrow \mathcal{G}$ 
2: for each  $(p_1, p_2) \in \mathcal{P}$  do
3:    $i \leftarrow \text{FindIndex}(p_1, \mathcal{N})$ 
4:    $j \leftarrow \text{FindIndex}(p_2, \mathcal{N})$ 
5:   if  $i \neq -1$  and  $j \neq -1$  then
6:      $\mathcal{G}'[i] \leftarrow \mathcal{G}'[i] \setminus \{j\}$ 
7:      $\mathcal{G}'[j] \leftarrow \mathcal{G}'[j] \setminus \{i\}$ 
8:   end if
9: end for
10: return  $\mathcal{G}'$ 
```

5.2.1 Complexity Analysis

This subsection analyzes the computational complexity of the line detection pipeline. Let H, W denote the image dimensions, n the number of nodes (endpoints), r the disk radius for node regions, and P the number of skeleton pixels (typically $P \ll H \times W$).

Time Complexity: Table 5-1 summarizes the time complexity of each operation in the pipeline.

Operation	Complexity	Description
Node ID Map Construction	$O(n \cdot r^2)$	Each node marks πr^2 pixels
Single Node BFS	$O(P)$	Each skeleton pixel visited at most once
Complete Adjacency Build	$O(n \cdot P)$	BFS from each of n nodes
Self-Edge Removal	$O(m)$	m = number of endpoint pairs
Total	$O(n \cdot r^2 + n \cdot P)$	Dominated by BFS traversals

Table 5-1 Time complexity breakdown for Multi-Head BFS algorithm

For typical skeleton images where $P = O(H + W)$ (linear in image perimeter), the overall complexity is:

$$T(n, H, W, r) = O(n \cdot r^2 + n \cdot (H + W)) = O(n \cdot (r^2 + H + W))$$

Space Complexity: Table 5-2 details the memory requirements of the algorithm.

Data Structure	Space	Description
Skeleton Image S	$O(H \times W)$	Binary image storage
Node ID Map M	$O(H \times W)$	Integer array for node mapping
Visited Array V	$O(H \times W)$	Per-BFS boolean array
BFS Queue Q	$O(P)$	Maximum skeleton pixels in queue
Adjacency $\mathcal{G}$	$O(n + E)$	E = number of edges
Disk Offsets $\mathcal{D}$	$O(r^2)$	Precomputed disk mask
Total	$O(H \times W + n + E)$	Dominated by image arrays

Table 5-2 Space complexity breakdown for Multi-Head BFS algorithm

The visited array can be reused across BFS calls with a reset, or alternatively, a hash set can be used for sparse skeletons to reduce visited storage to $O(P)$.

5.3 Frontend and Backend Implementation

The frontend system has been realized using React.js, Chart.js and relevant packages like react-draggable and KiCad Symbol Library. Likewise, the backend of the system exposes a API that manages the complete lifecycle of circuit simulations. The API provides endpoints for submitting SPICE netlists, validating circuit syntax, and executing simulations using different analysis modes such as transient, AC, and DC analysis.

Each simulation request is assigned a unique identifier, enabling asynchronous execution and progress tracking through dedicated status endpoints. The API also supports retrieval of detailed simulation results, including time-domain voltage and current data, as well as administrative operations such as stopping running simulations, listing historical simulations, and deleting completed or failed runs. Standard Hypertext Transfer Protocol (HTTP) status codes and structured JSON responses are used throughout the API to ensure consistency, robust error handling, and seamless integration with the frontend application.

5.3.1 Frontend-Backend Interaction

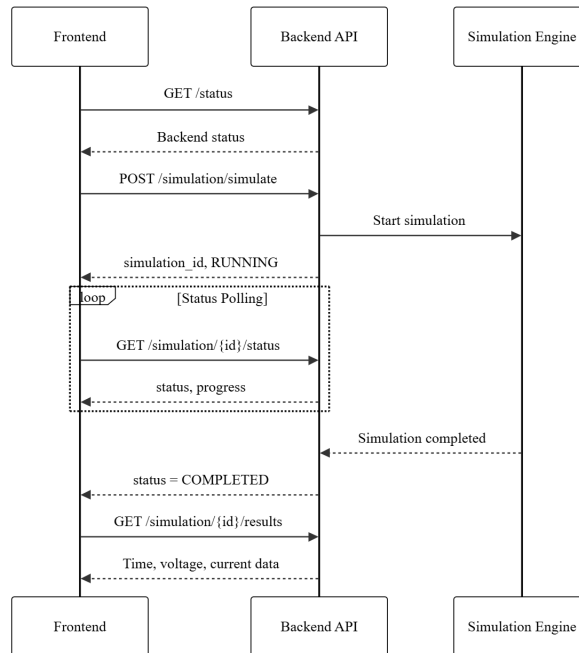


Figure 5-1 Interaction Diagram for the Web Application

The frontend communicates with the backend through HTTP requests to the documented endpoints. User inputs from the editable circuit schematic are converted into SPICE netlists and submitted to the simulation endpoint. The frontend periodically polls the status endpoint to track simulation progress, and upon completion, retrieves and visualizes the results using Chart.js for graph generation and React components for interactive display.

5.4 Circuit Simulation using PySpice

is a interface to (an open-source simulation engine) that takes in the netlist of the circuit and returns the data required to visualize graphs related to the circuit. The integration allows the backend to programmatically define circuits, run simulations, and extract result data in a structured format suitable for visualization and analysis.

5.4.1 Netlist to Simulation Results Conversion

The process of converting a circuit netlist to simulation results involves several key steps: parsing the netlist, initializing the simulation environment, configuring analysis parameters, executing the simulation, and extracting the resulting voltage and current data in a format compatible with the frontend visualization system.

Algorithm 15 Resistor-Capacitor (RC) Circuit Transient Simulation and Data Export

Require: Supply voltage V_s , resistance R , capacitance C , simulation step Δt , simulation duration T

Ensure: Comma-Separated Values (CSV) file containing transient voltage response

Initialize SPICE circuit named *RC Charging Circuit*

Enable logging for simulation diagnostics

Define circuit components

Add DC voltage source of magnitude V_s between input node and ground

Connect resistor R between input node and node $n1$

Connect capacitor C between node $n1$ and ground

Initialize simulator with nominal temperature

Perform transient analysis with step size Δt up to time T

Extract time vector from transient analysis

Extract voltage waveform at node $n1$

Create data table mapping time to voltage

Export data table to CSV file

return Generated CSV file

6 RESULTS AND ANALYSIS

6.1 Comparative Analysis of Object Detection using YOLOv5 and YOLOv7

6.1.1 Experimental Setup

A comparative study using YOLOv5 and YOLOv7 to detect and classify hand-drawn electronic circuit components was conducted to determine which model is better-suited for the needs of this project. The dataset consisted of 61 classes, including resistors, capacitors (polarized and unpolarized), diodes, transistors (Bipolar Junction Transistor (BJT) and Field-Effect Transistor (FET)), integrated circuits, switches, and more. To ensure a fair comparison, both models were trained and validated on identical data splits.

6.1.2 Training Progress and Loss Curves

While training, loss metrics (bounding box loss, objectness loss, and the classification loss) were tracked. The training curves show how these metrics evolved over time. No substantial difference was seen between the loss metrics of YOLOv5 and v7.

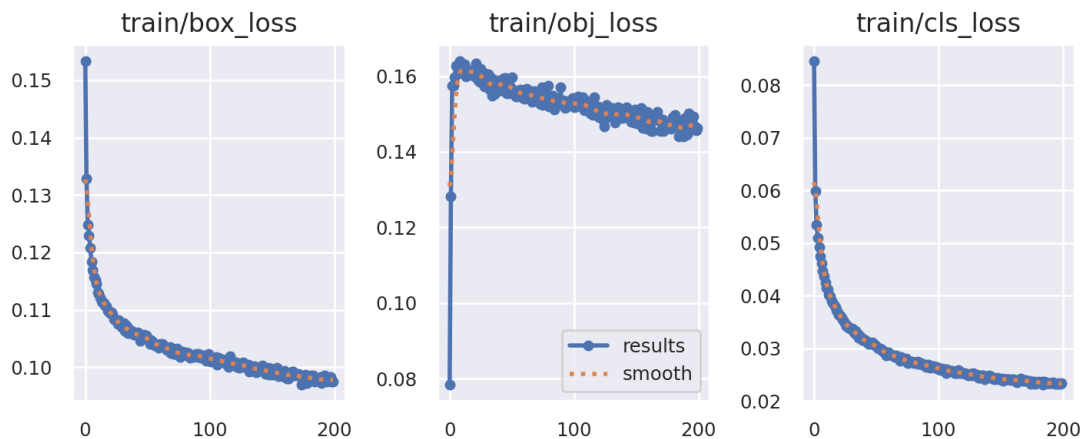


Figure 6-1 Loss Curves for YOLOv5

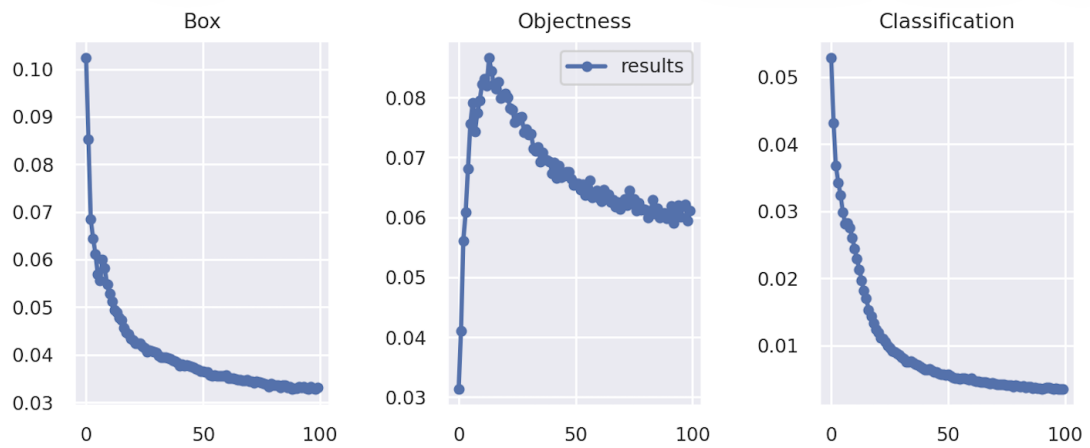


Figure 6-2 Loss Curves for YOLOv7

6.1.3 Performance Metrics

mAP vs Number of Epochs

In order to monitor the progress in models' ability to recognize components as the training progressed, both mAP@0.5 and mAP@0.5:0.95 were plotted across all epochs. These curves depict how quickly and effectively each model learns over time.

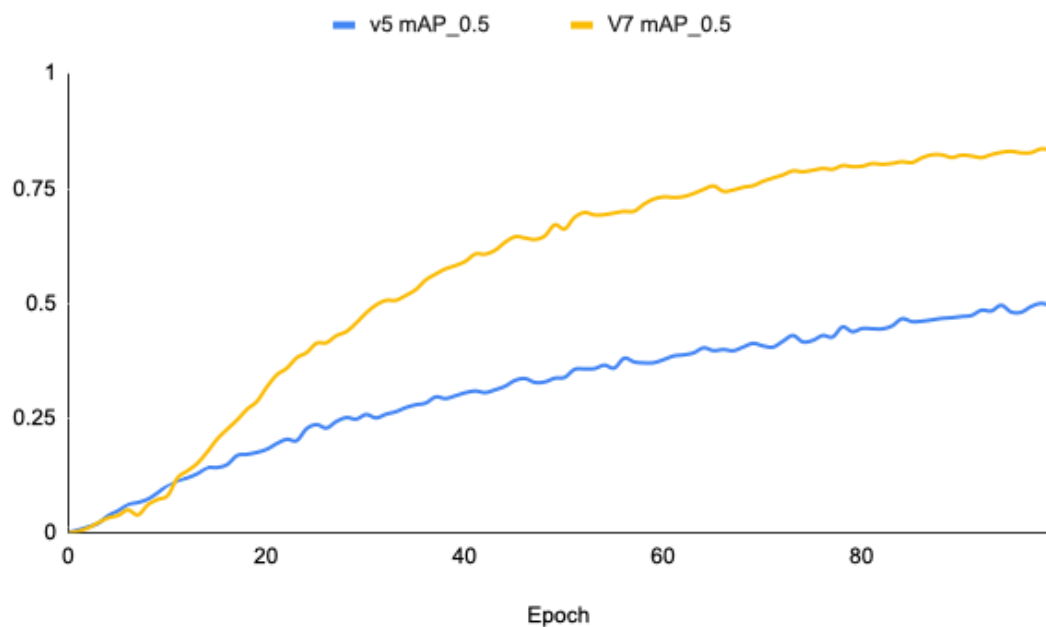


Figure 6-3 mAP@0.5 vs Epochs for YOLOv5 and YOLOv7

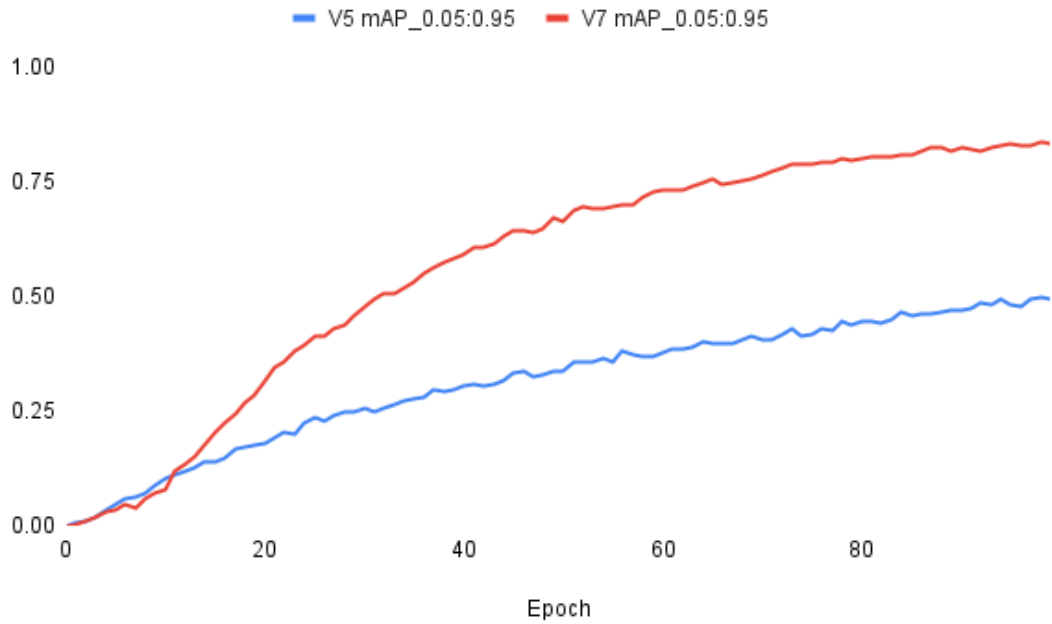


Figure 6-4 mAP@0.5:0.95 vs Epochs for YOLOv5 and YOLOv7

It is clear from both Figure 6-3 and Figure 6-4 that the mAP scores for YOLOv7 are significantly better than that for YOLOv5. Likewise, the loss curves, as shown in Figures 6-1 and 6-2, demonstrate a steeper curve in the case of YOLOv7.

Precision, Recall, and F1 Score vs Confidence

The following graphs demonstrate the model performance under varying confidence thresholds:

Precision vs Confidence

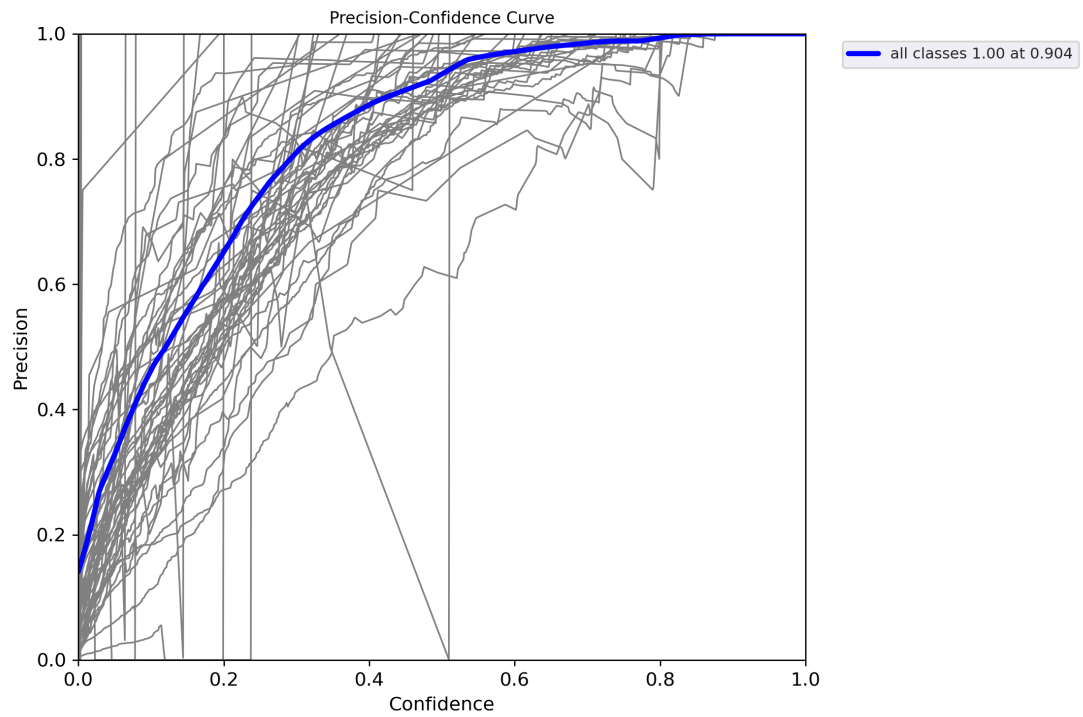


Figure 6-5 Precision-Confidence Curve for YOLOv5

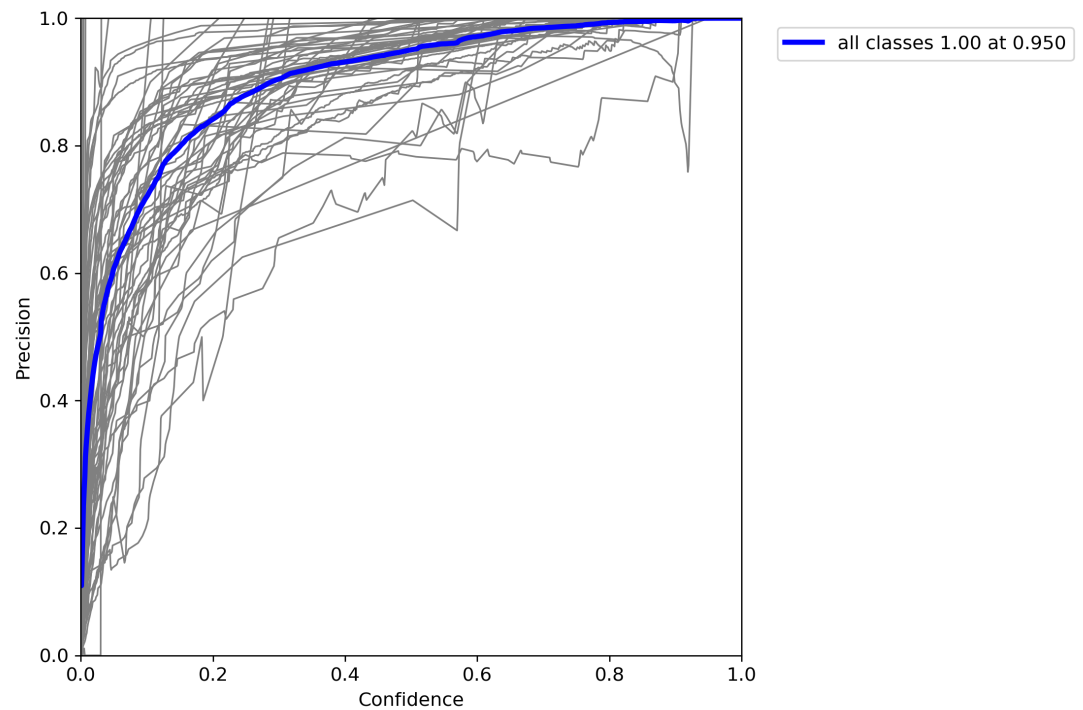


Figure 6-6 Precision-Confidence Curve for YOLOv7

Figure 6-5 and Figure 6-6 demonstrate the Precision-Confidence Curve for YOLOv5 and YOLOv7 respectively. The area under this curve informs how consistently precise the model is across all confidence thresholds. For a large area under the curve, as in the case of YOLOv7, the model maintains good precision even when threshold changes; however, for a smaller area, precision collapses quickly when threshold is relaxed.

Recall vs Confidence

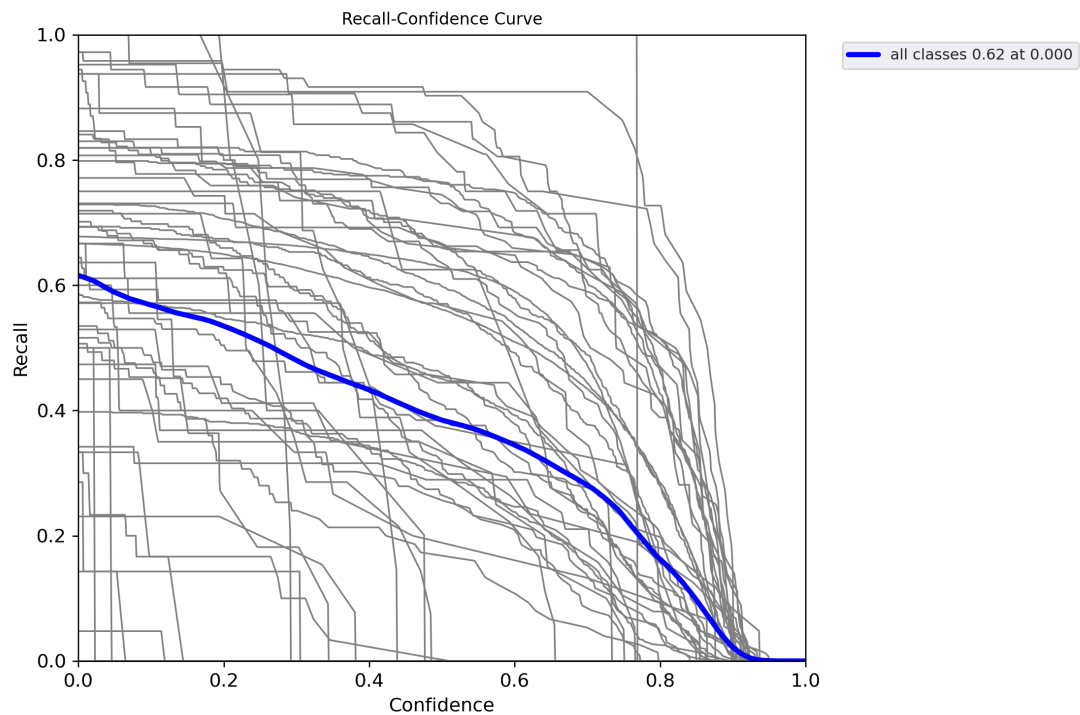


Figure 6-7 Recall-Confidence Curve for YOLOv5

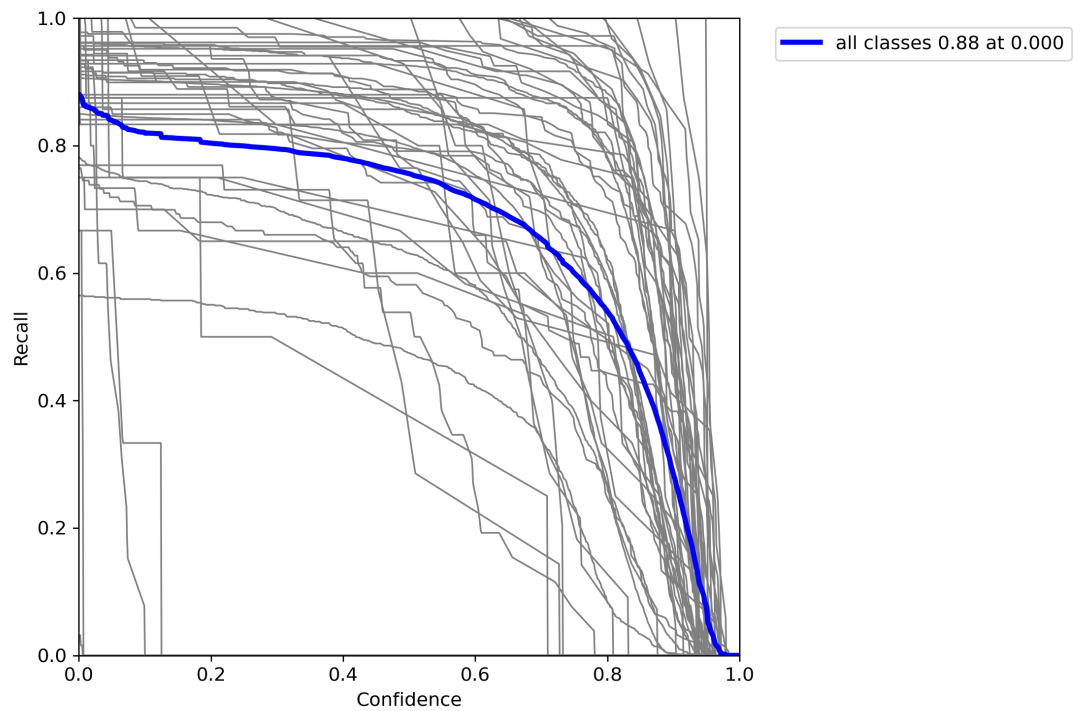


Figure 6-8 Recall-Confidence Curve for YOLOv7

Figure 6-7 and Figure 6-8 demonstrate the Recall-Confidence Curve for YOLOv5 and YOLOv7 respectively. The area under this curve informs how well the model preserves recall while increasing certainty. For a large area, the recall stays high even with strict thresholds.

F1 Score vs Confidence

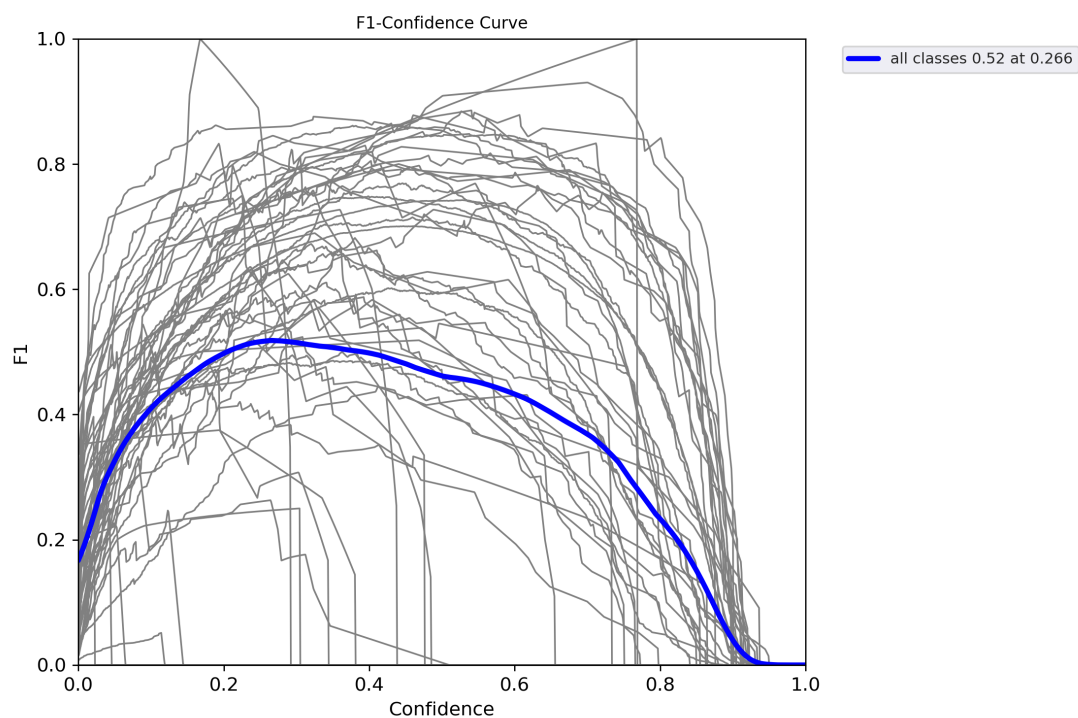


Figure 6-9 F1 Score-Confidence Curve for YOLOv5

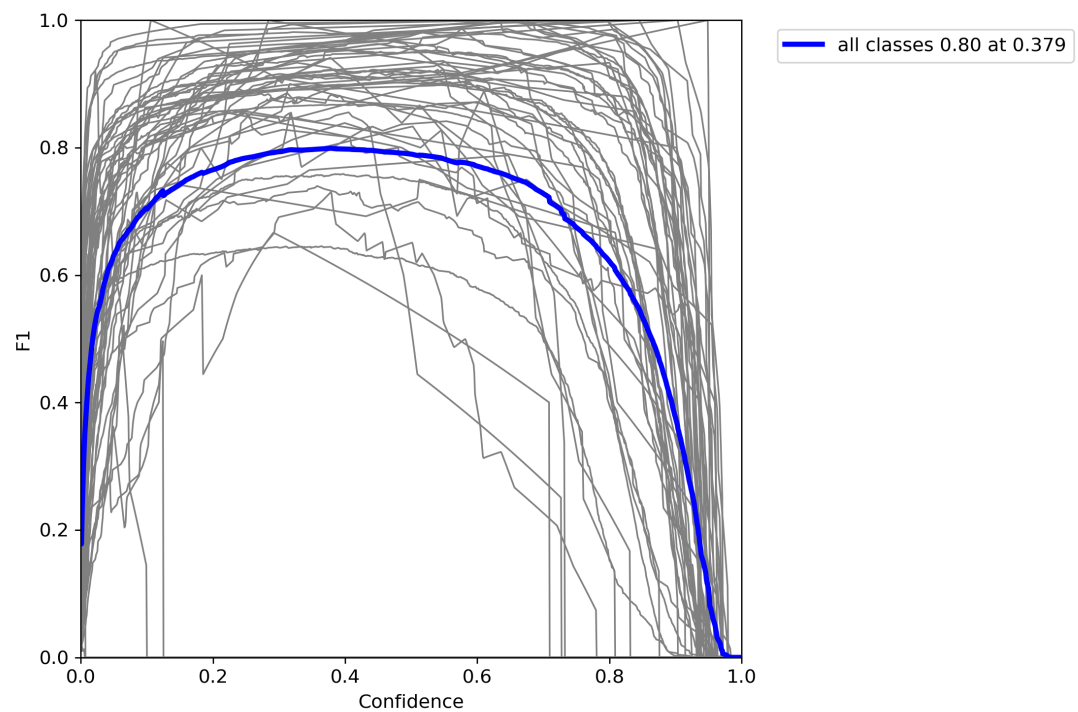


Figure 6-10 F1 Score-Confidence Curve for YOLOv7

Figure 6-9 and Figure 6-10 demonstrate the F1 Score-Confidence Curve for YOLOv5 and YOLOv7 respectively. The larger the area under this curve, the higher the robustness of the precision-recall trade-off across thresholds.

Precision-Recall Curve

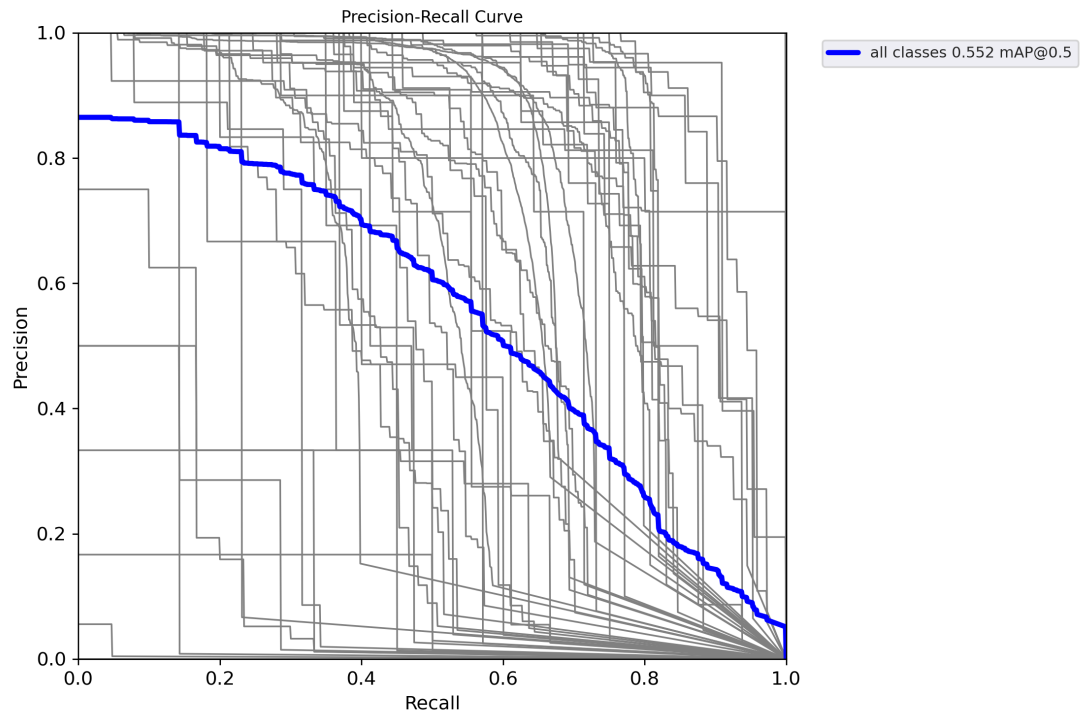


Figure 6-11 Precision-Recall Curve for YOLOv5

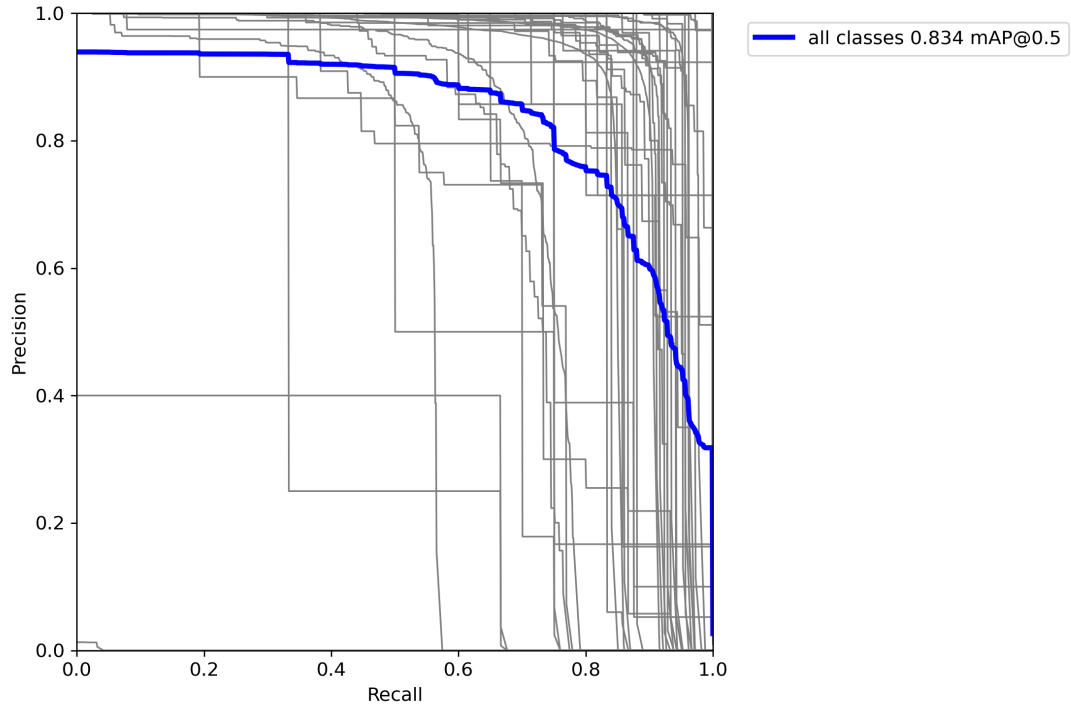


Figure 6-12 Precision-Recall Curve for YOLOv7

Figure 6-11 and Figure 6-12 demonstrate the Precision-Recall Curve for YOLOv5 and YOLOv7 respectively. The area under this curve gives the average precision (AP) which informs how good the model is at ranking positive samples above negative ones. The area under each of the curves in these figures is higher for YOLOv7 in comparison to YOLOv5. This is expected behavior and proves YOLOv7 is better than YOLOv5 when it comes to performance. Although YOLOv7 is computationally more intensive and generally heavier than YOLOv5, this is a trade-off willing to be accepted to gain performance.

6.1.4 Confusion Matrix

To assess per-class performance, confusion matrices were generated using the validation set.

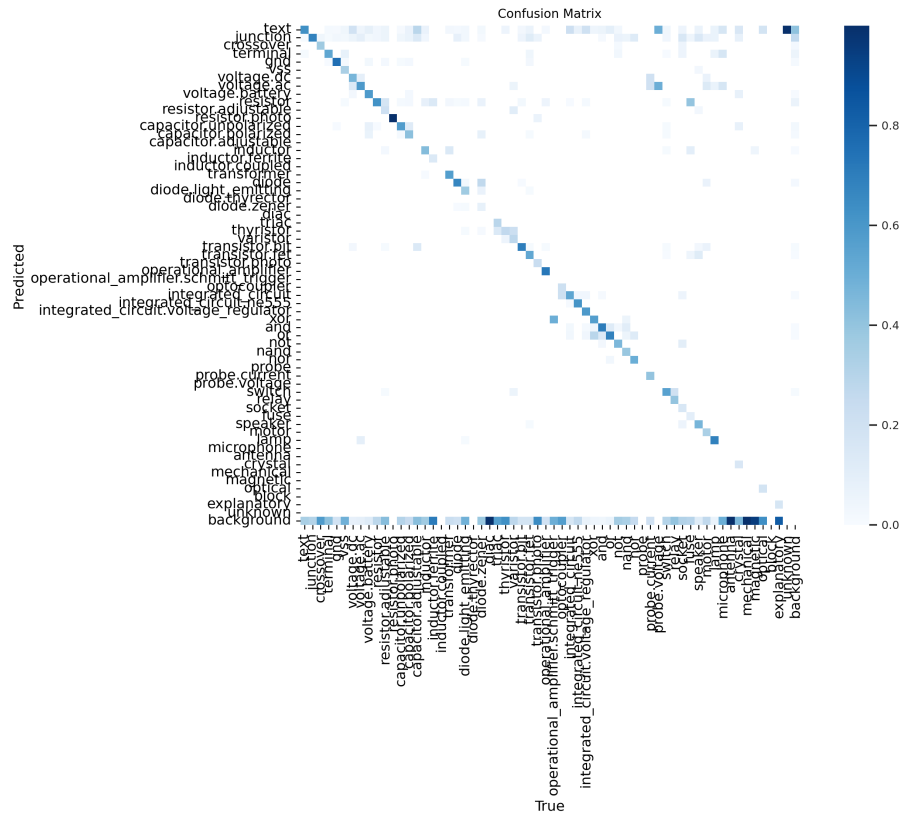


Figure 6-13 Confusion Matrix for YOLOv5

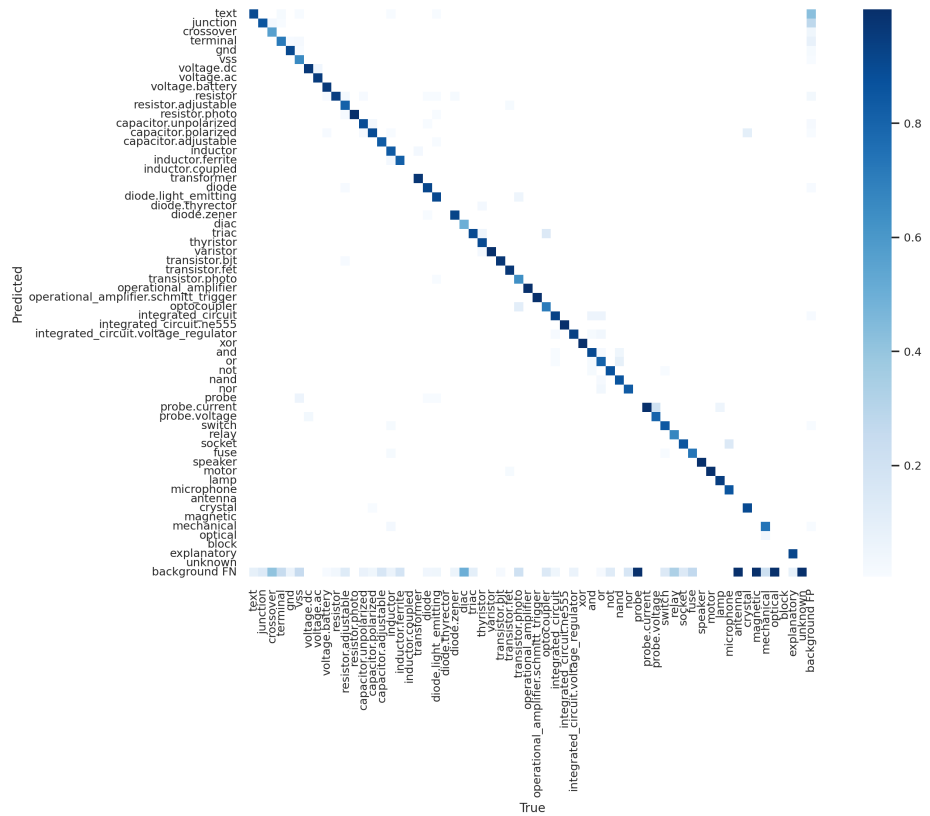


Figure 6-14 Confusion Matrix for YOLOv7

Figure 6-13 and Figure 6-14 demonstrate the confusion matrices for YOLOv5 and YOLOv7 respectively. In comparison, the confusion matrix for YOLOv7 has a very dark main diagonal and very light off-diagonals, which means that the model exhibits strong class separability with minimal inter-class confusion.

6.1.5 Quantitative Results Comparison

Final epoch results show YOLOv7 outperformed YOLOv5 across all major metrics.

Table 6-1 Comparison of Performance Metrics for YOLOv5 and YOLOv7

Metric	YOLOv5	YOLOv7
mAP@0.5	0.61	0.83
mAP@0.5:0.95	0.41	0.60
Precision	0.84	0.93
Recall	0.62	0.78
Epochs Trained	200	100

Overall, YOLOv7 showed a clear advantage over YOLOv5 in our experiments. It not only achieved higher values for precision, recall, and mean average precision but also required fewer epochs to converge. YOLOv7's confusion matrix reflected better class separation and fewer misclassifications. The improvements in F1 score and overall stability of predictions across confidence thresholds make YOLOv7 a more robust choice for circuit diagram interpretation.

6.2 OCR Results

The out-of-the-box default implementation without fine-tuning was able to detect some of the component values from the circuit; however, the results left a lot to be desired. This made it clear that proper training of the recognition model is absolutely essential for this project.

After training the text recognition model from scratch, results were much improved. Two randomly picked images from the dataset are shown below after passing through the detection (YOLOv7) and recognition (OCR) models.

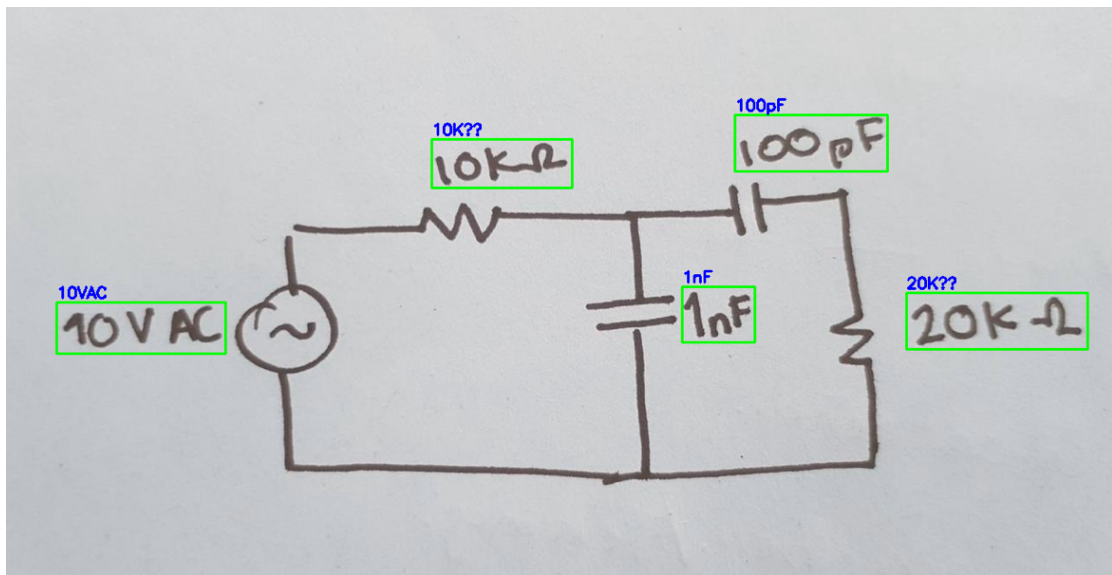


Figure 6-15 Text Detection and Recognition (i)

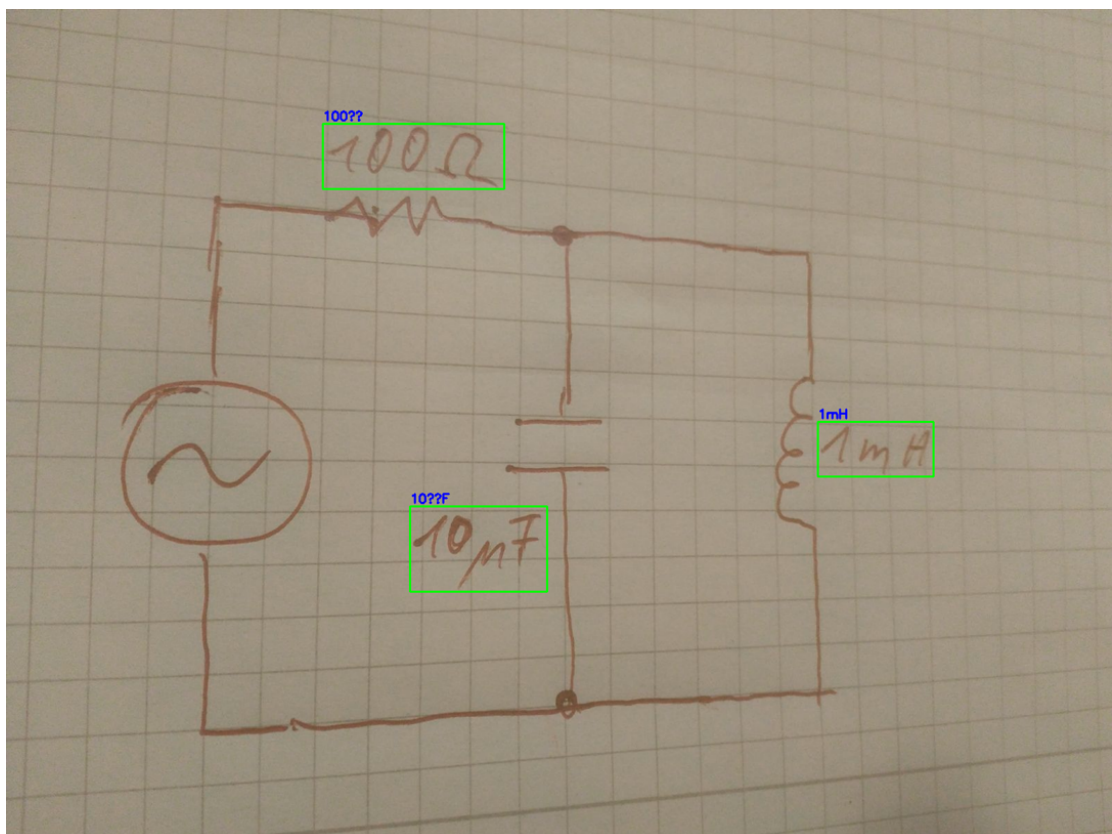


Figure 6-16 Text Detection and Recognition (ii)

It is to be noted that the “??” shown in Figure 6-15 and Figure 6-16 is a limitation of the picture-rendering library used (OpenCV, Matplotlib), rather than a limitation of the OCR model. The OCR model accurately identifies symbols like μ (micro).

6.2.1 Training Loss Curve

Figure 6-17 shows that the training loss decreases consistently throughout the 50 epochs, indicating the model is effectively memorizing the training data. However, the validation loss only drops until around 15-20 epochs before gradually increasing, indicating overfitting. Overfitting is an issue to be eradicated using extensive training using diverse datasets (either by addition of other datasets in training and/or augmentation). If the performance is considered satisfactory after training for only 15-20 epochs, early stopping may also be used.

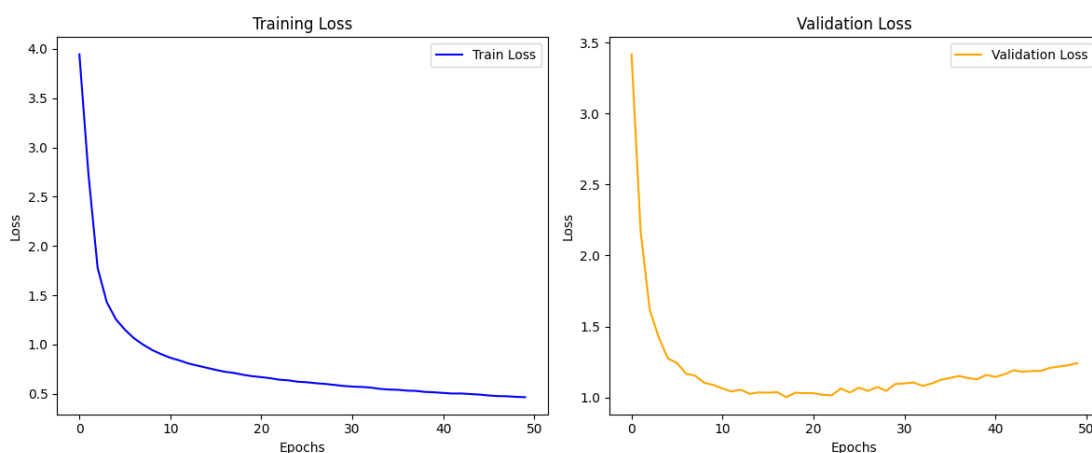


Figure 6-17 Training Loss vs Epochs (OCR)

6.2.2 OCR Prediction

The predictions made by the OCR model for a random sample of test data is shown in Figure 6-18. The OCR model accurately recognizes the easily discernible text but is fallible in some cases. However, in this random sample of images, all the problem images (i.e. the ones OCR cannot recognize) are indiscernible to human eyes as well. For instance, the input image samples for GT: D5 and GT: 31 are completely black suggesting a preprocessing or cropping bug rather than a fault in the model.

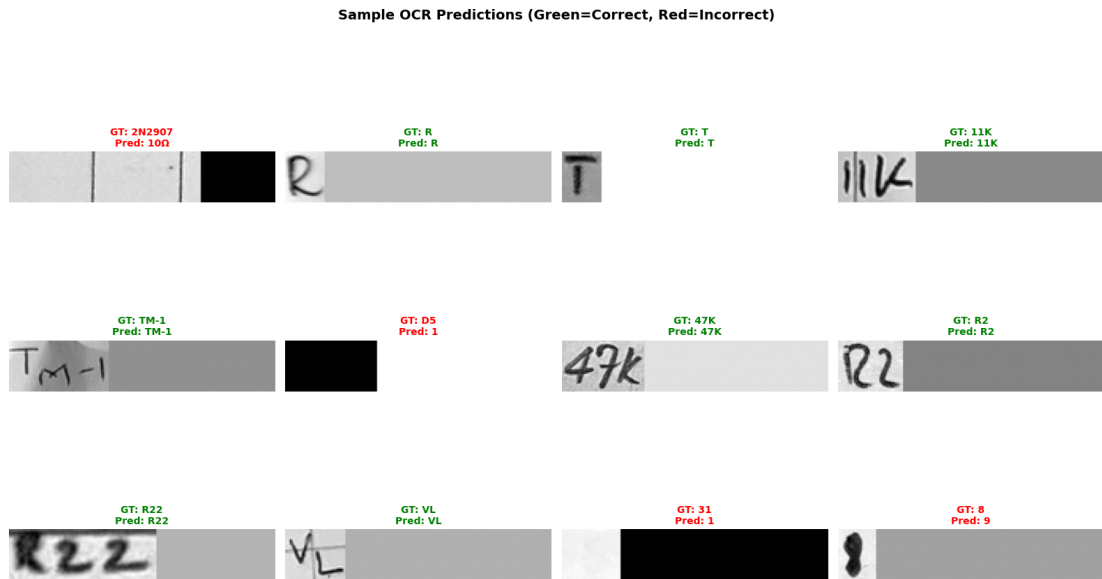


Figure 6-18 OCR Model Predictions

6.2.3 OCR Performance Analysis

Figure 6-19 is a combination of images showing the performance metrics for the OCR model. The accuracy is 65.96% and while this may seem low, the sample images reveal that accuracy might have taken a hit due to data corruption in the cropping process or preprocessing. Once this bug is somewhat fixed, accuracy is expected to rise. That being said, to improve the accuracy further, the task of refining the model, regardless of data corruption will continue. Likewise, the CER of 0.25 means that on average, 1 out of every 4 characters is predicted incorrectly. However, the histogram shows a notable bump at 1.0 (complete failure) confirming the presence of "garbage" inputs (like black boxes, as seen in the sample images). WER is shown to be 0.34. Its distribution is inherently binary because the model either gets the full component value right (WER being 0) or gets it wrong (WER = 1).

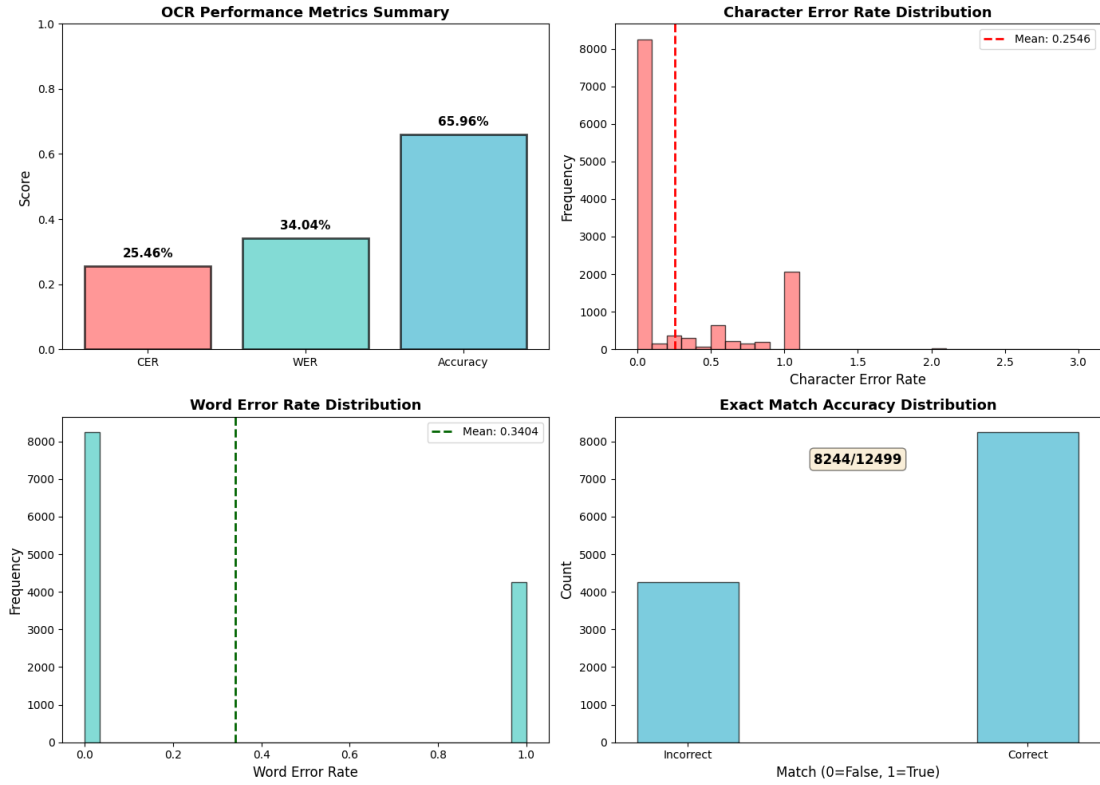


Figure 6-19 OCR Performance Metrics and their Distributions

6.3 Junction-Based Wire Detection

This section presents the experimental results obtained from the proposed junction-based wire detection pipeline applied to circuit diagram images. Each processing stage is evaluated independently and supported with quantitative measurements and visual evidence.

6.3.1 Input Image and Preprocessing

The experiment begins with a high-resolution scanned circuit diagram used as the input image. The original image dimensions are reported in terms of width and height in pixels to establish spatial scale consistency throughout the pipeline.

To enhance structural features and suppress background noise, the input image is converted to grayscale. Otsu's global thresholding method is then applied to automatically compute an optimal threshold value that separates foreground circuit elements from the background. The resulting binary image preserves wires, junctions, and components while eliminating illumination variations.

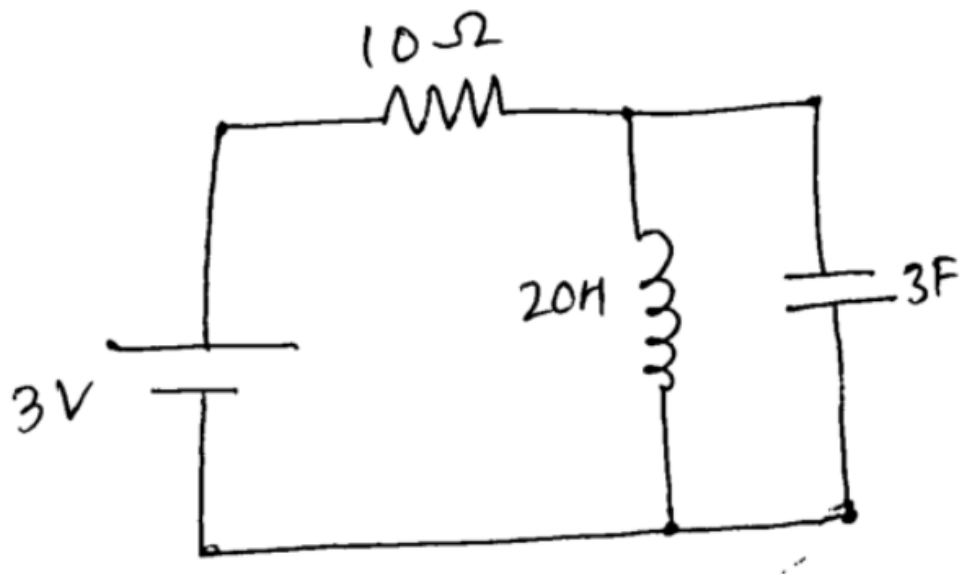


Figure 6-20 Otsu's Global Thresholding

6.3.2 Skeletonization

Morphological skeletonization is applied to the binary image to reduce all wire segments to single pixel-wide centerlines. This transformation significantly reduces the number of foreground pixels while preserving the topological connectivity of the circuit network.

The reduction in foreground pixel count before and after skeletonization is recorded to quantify structural simplification. Despite aggressive thinning, wire continuity and junction connectivity remain intact, making the skeleton suitable for graph-based analysis.

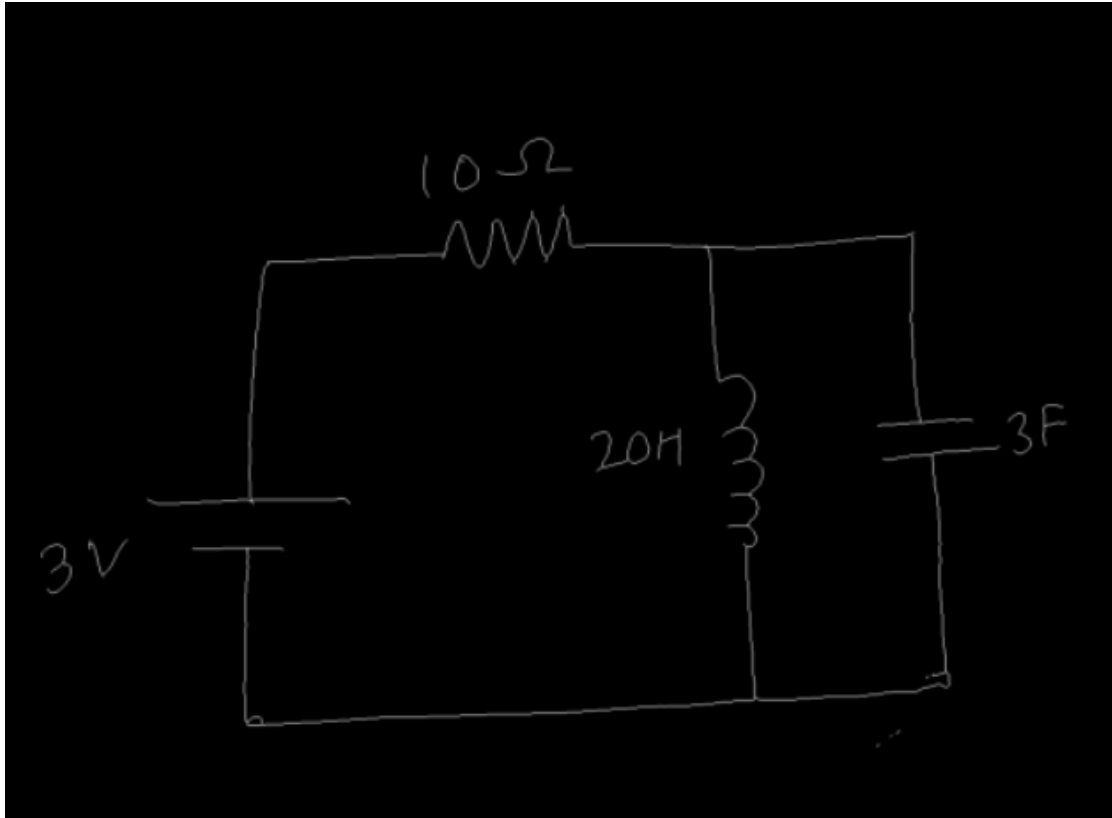


Figure 6-21 Skeletonization

6.3.3 Component Removal and Junction Isolation

Object detection is performed using a trained YOLO model to identify and localize elements in the circuit diagram. Detected objects are classified as follows: Class 0 corresponds to text annotations, Class 1 corresponds to junctions, and all remaining classes correspond to circuit components.

The total number of detected objects per class is reported. To isolate only the wire network, bounding boxes associated with non-junction classes are removed from the skeleton image. This step eliminates component shapes while preserving junction points and interconnecting wires.

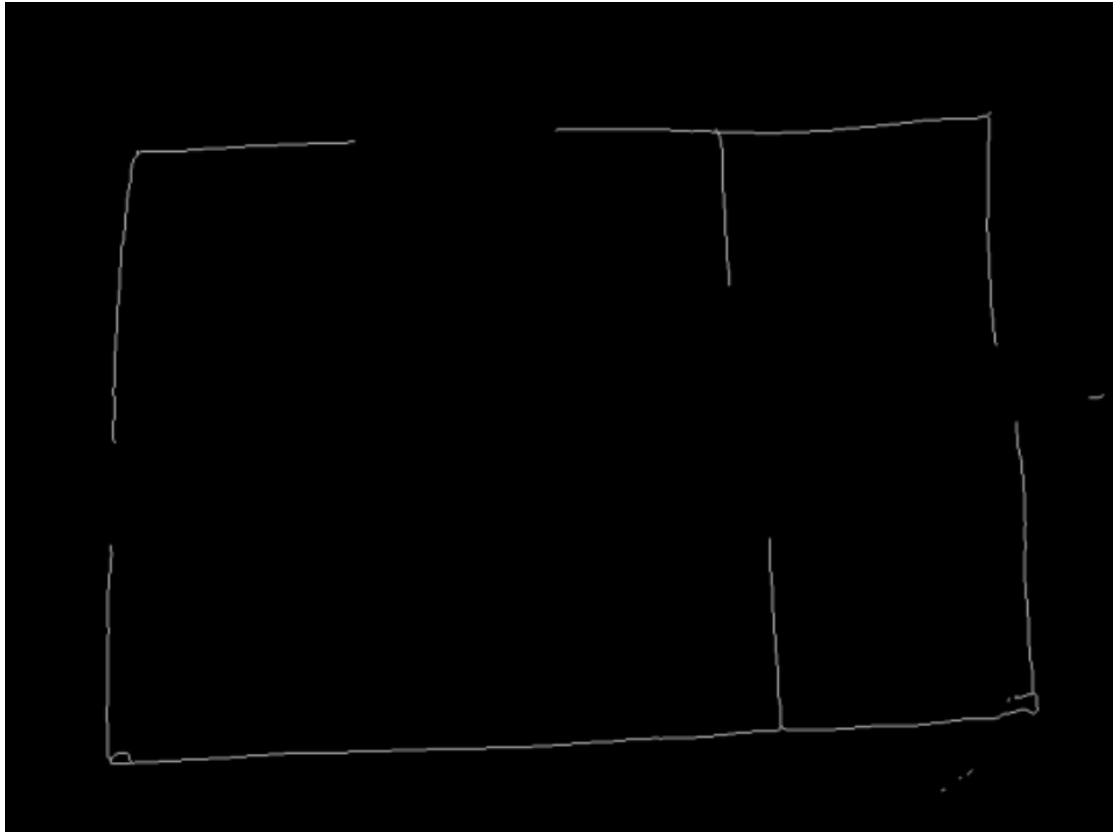


Figure 6-22 Component Removal

6.3.4 Junction Extraction and Wire Classification

All detected junctions belonging to Class 1 (junctions) are extracted and their coordinates are converted from normalized YOLO format to pixel coordinates.

Each junction pair is classified based on geometric alignment. Pairs aligned horizontally are labeled as horizontal wires, pairs aligned vertically are labeled as vertical wires, and diagonally aligned pairs are rejected. The total number of accepted and rejected pairs is reported. Sample coordinate transformations from normalized values to pixel coordinates are also presented.

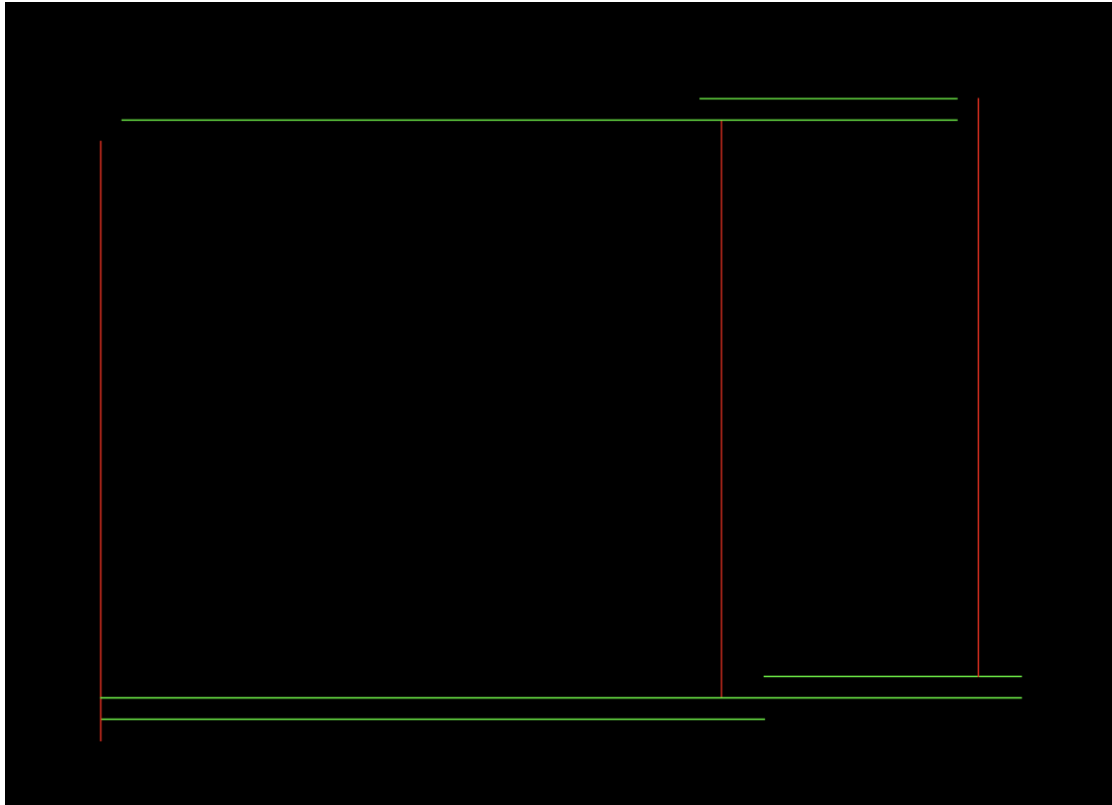


Figure 6-23 Wire Classification

6.3.5 Segment Consolidation Through Merging

Initial wire detection may produce fragmented segments representing portions of the same physical wire. To address this, a merging algorithm is applied based on axis alignment tolerance of 16 pixels and overlapping or adjacent segment conditions.

The total number of raw wire segments before merging and the consolidated count after merging are reported.

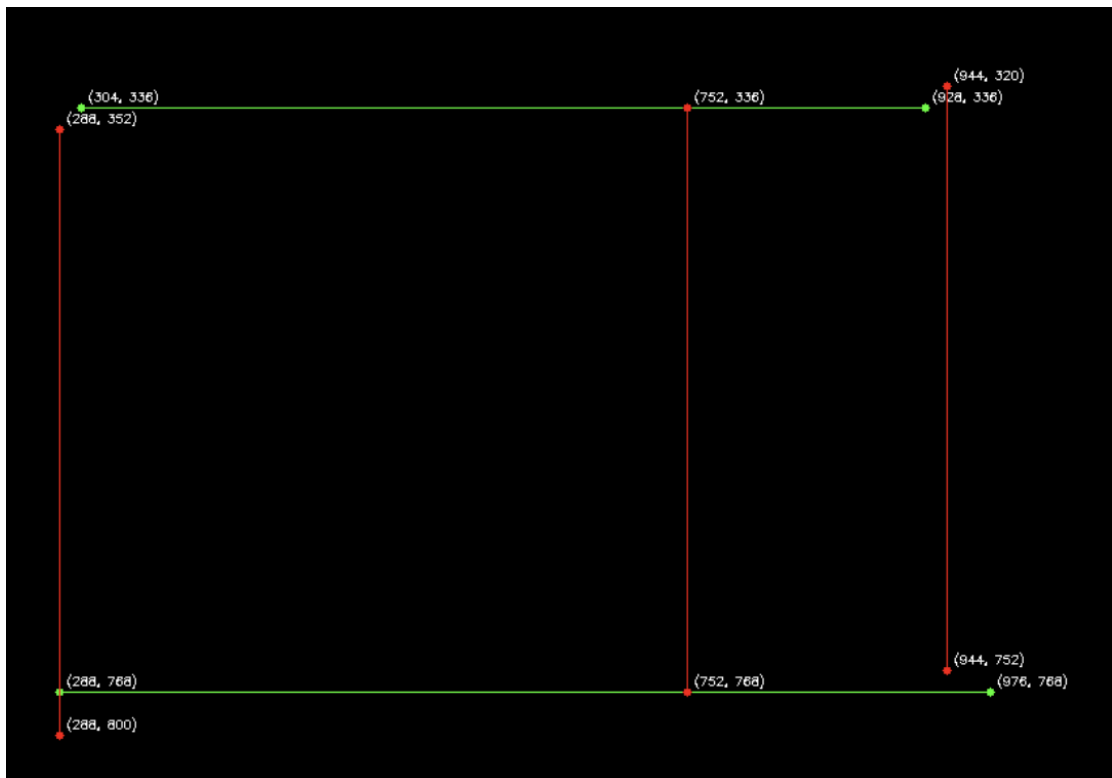


Figure 6-24 Segment Consolidation Through Merging

6.3.6 Component Bounding Box Overlay

To enhance visualization and verify correct detection, component bounding boxes detected by YOLO are overlaid on the original circuit diagram. This overlay confirms that all circuit elements have been properly identified and localized, with bounding boxes accurately delineating component boundaries.

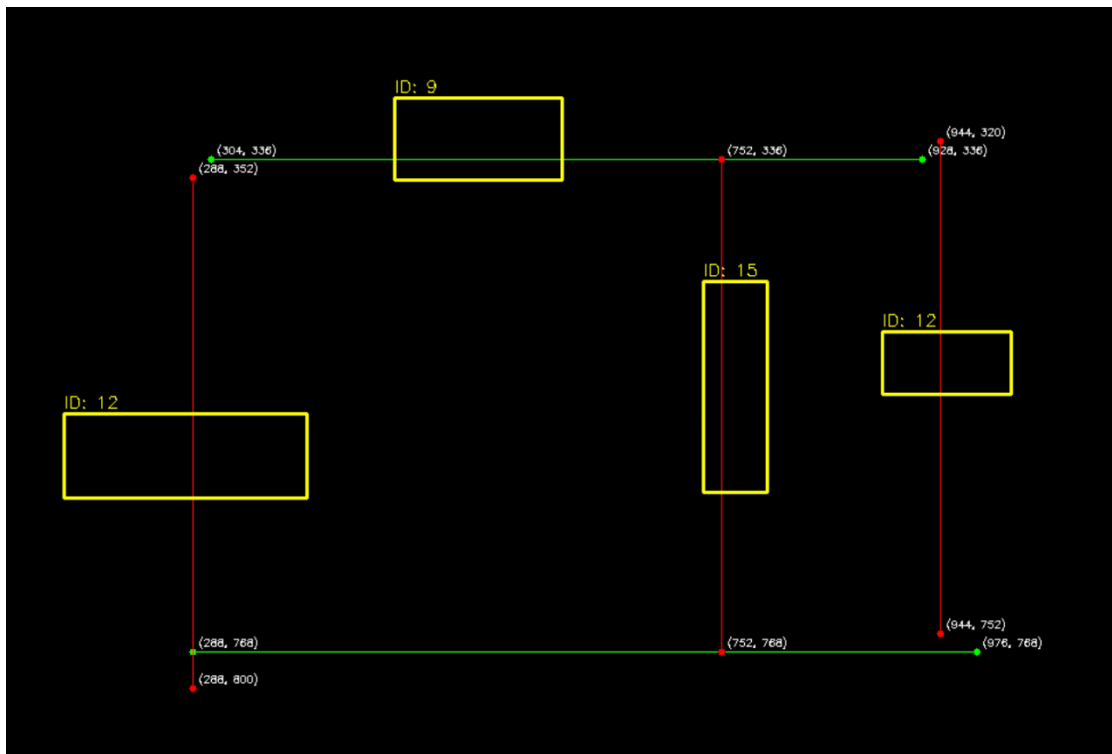


Figure 6-25 Bounding Box Overlay

6.3.7 Wire-Component Intersection Detection

Intersection points between wires and component bounding box edges are computed to identify electrical connectivity. The total number of intersection points is reported along with a directional breakdown for horizontal and vertical wires intersecting specific component edges.

These intersections form the basis for reconstructing circuit connectivity graphs and netlists.

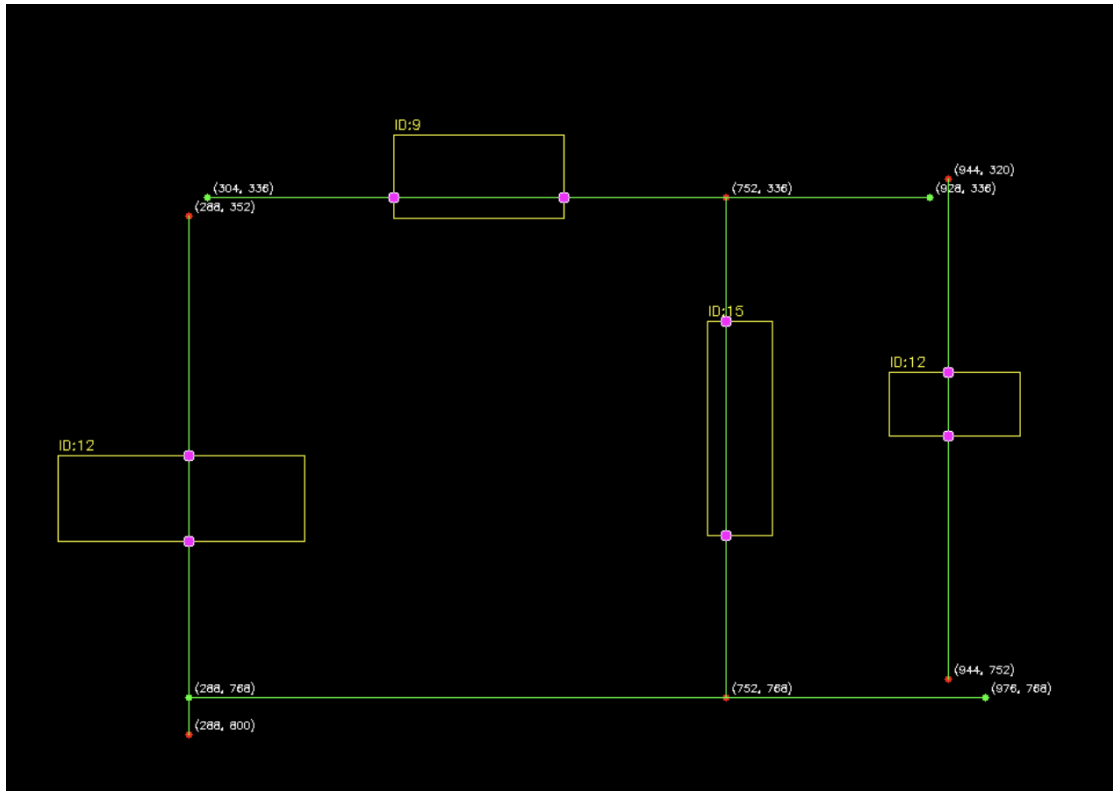


Figure 6-26 Wire-Component Intersection Detection

6.4 Line Detection via Path Finding Visualization

The algorithm produces visual outputs that demonstrate the effectiveness of the endpoint detection and graph construction pipeline. The following subsections present the key visualizations obtained from applying the Multi-Head BFS algorithm to the skeletonized circuit image.

6.4.1 Skeletonized Image with Detected Endpoints

Figure 6-27 displays the skeletonized image with all detected endpoints overlaid. The skeleton representation captures the one-pixel-wide structure of the circuit lines after morphological processing. Red circles indicate the detected endpoints, which were identified using the bounding box intersection method for line segments and center-snapping for junction points. Green rectangles show the original YOLO bounding boxes used for endpoint localization. This visualization confirms that the endpoint detection algorithm successfully identifies connection points at the boundaries of detected objects.

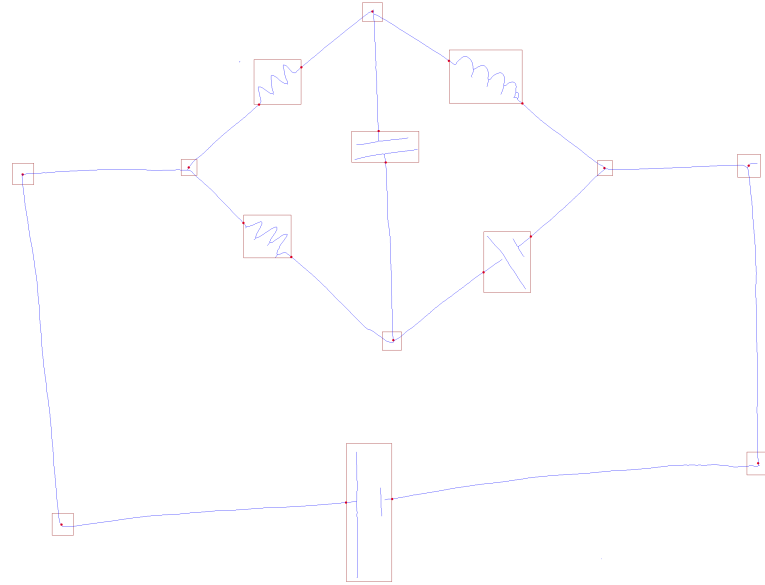


Figure 6-27 Skeletonized image with detected endpoints

6.4.2 Adjacency Graph Visualization

Figure 6-28 presents the final adjacency graph constructed using the Multi-Head BFS algorithm. Each node corresponds to a detected endpoint from the previous stage. Blue lines connecting the nodes represent edges in the graph, indicating skeleton connectivity between endpoints. The graph has been processed to remove self-edges (connections between endpoints belonging to the same detected object), ensuring that only meaningful inter-component connections are preserved. This graph structure enables downstream applications such as circuit topology analysis and netlist extraction.

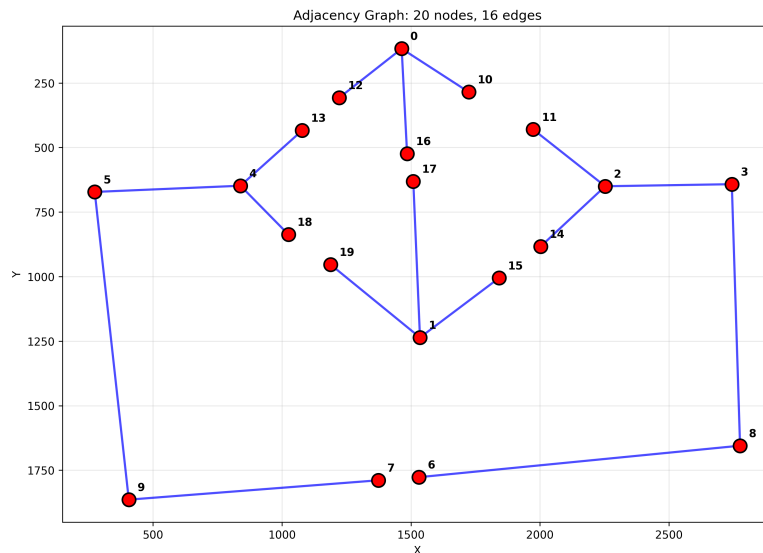


Figure 6-28 Adjacency graph constructed using Multi-Head BFS algorithm

6.5 Frontend Visualization

The frontend of the web application provides an interactive environment for circuit design, simulation, and analysis. It enables users to construct electrical circuits using a drag-and-drop schematic editor with grid-based alignment, snap-to-grid functionality, and real-time wire routing. Components such as passive elements, active devices, and sources can be configured through a properties panel that allows precise control over electrical parameters and placement.

The system integrates simulation controls for transient analysis, allowing users to define time parameters, execute simulations, and visualize results through dynamically rendered voltage-time plots. Measurement probes support real-time signal monitoring with automated computation of key metrics, including maximum, minimum, peak-to-peak range, and Root Mean Square (RMS) values. Additionally, the frontend supports exporting simulation data and visualizations in standard formats, enabling seamless documentation and further offline analysis.

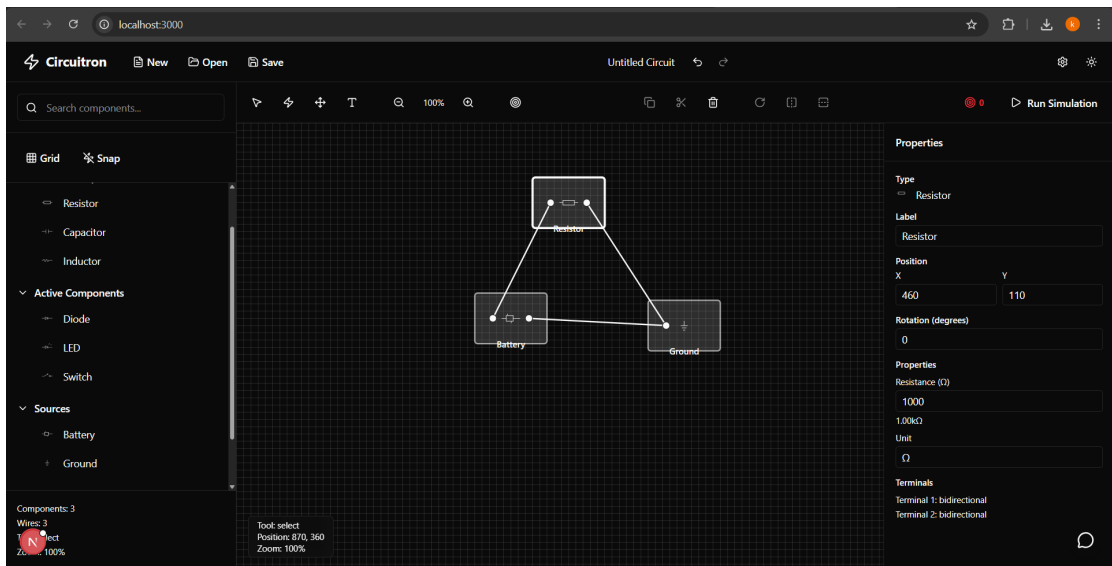


Figure 6-29 Digitally Drawn Circuit

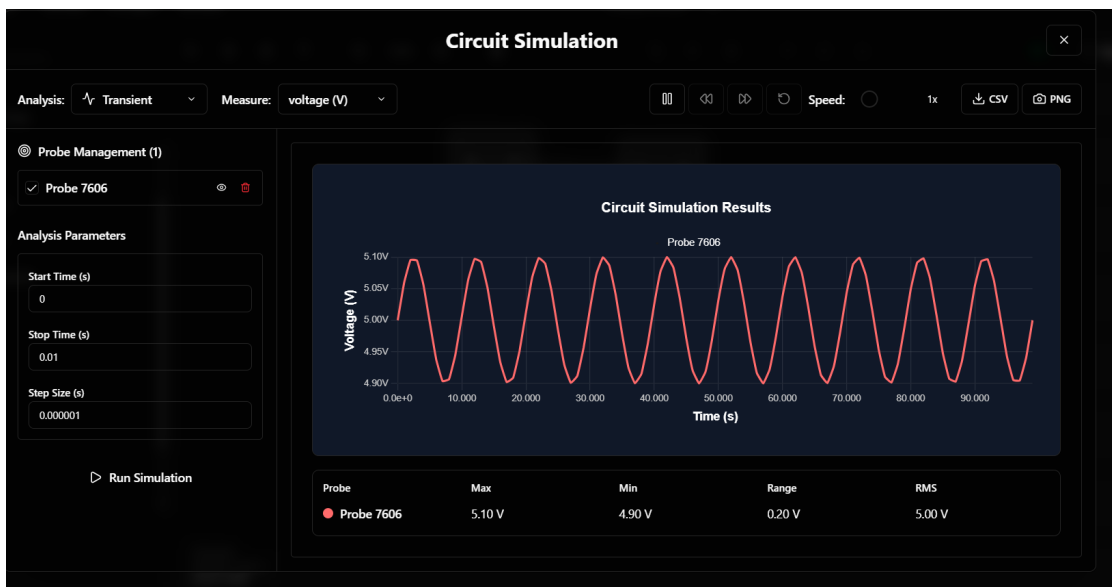


Figure 6-30 Example Circuit Simulation Result

7 REMAINING TASKS

As of yet, the individual component/text detection and extraction elements along with node detection and wire connectivity seem to be working. Component detection with fine-tuned YOLOv7 model, text extraction using custom OCR model trained from scratch (working in tandem), and path finding-based wire detection have been achieved. Moreover, these separate modules have been successfully integrated to build a complete project pipeline that takes an image of a circuit and gives out the simulation graphs after selected simulation type (transient analysis, constant voltage analysis, etc). While work on improving the accuracy and robustness of the models will definitely continue in the background until the target values for metrics are reached, the primary goal is to add a few user-focused features. LLM-integration (Gemini) is a major task remaining. Such an LLM-based question-and-answer feature enables users to interact with the system, ask questions about the circuit, and receive direct feedback and analysis based on the simulation results. Integration of authentication and database in the project pipeline is the next step as a facilitating step for users to save a particular circuit for future use. Other minor remaining tasks include effective integration of the built APIs with the frontend of the web-app, and general refinement duties. This leaves the hosting part. For hosting, if possible, a free service (Vercel or Python Anywhere) may be used, or a paid service (Microsoft Azure) can be used.

7.1 Summary of Remaining Tasks

Table 7-1 Summary of Remaining Tasks

SN	Modules	Remarks
1	LLM Integration	Gemini integration for question-and-answer feature
2	Authentication and Database	For saving and retrieving circuits for future use
3	API and Frontend Integration	Effective integration of built APIs with web-app
4	System Refinement	General refinement and optimization duties
5	Hosting and Deployment	Free (Vercel, PythonAnywhere) or paid (Azure) services

8 APPENDICES

Appendix A: Budget Estimation

The total budget required for the successful completion of the project is estimated to be NPR 41,160. This budget covers various aspects of the project, including computation resources, cloud services, IEEE membership, and miscellaneous expenses.

Cost Estimation Breakdown

Table 8-1 Project Budget Breakdown

Category	Details	Estimated Cost (NPR)
Computation Requirements	Google Colab Pro (\$9.99 per month), NVIDIA T4 GPU (\$0.35 per hour), Lightning AI Credits	20,000
Cloud Service and Hosting	Domain Registration (\$12), Cloud Storage 100 GB (\$3 per month), Database hosting (\$30-\$50 per month)	4,000
IEEE Xplore Membership	Annual subscription for accessing research articles, journals	2,160
Miscellaneous Costs	Report printing, binding, API, etc.	15,000
Total		41,160

Appendix B: Gantt Chart

Table 8-2 Project Gantt Chart

PROCESS	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar
Research											
Implementation											
Testing											
Documentation											
Final Submission											
Deployment											

REFERENCES

- [1] R. R. Reddy and M. R. Panicker, “Hand-drawn electrical circuit recognition using object detection and node recognition,” *arXiv preprint arXiv:2106.11559*, Jan 2021, available: <https://arxiv.org/abs/2106.11559>.
- [2] A. E. Sertdemir, M. Besenk, T. Dalyan, Y. D. Gokdel, and E. Afacan, “From image to simulation: An ANN-based automatic circuit netlist generator (Img2Sim),” in *Proc. IEEE Int. Conf. on Synthesis, Modeling, Analysis and Simulation Methods and Applications to Circuit Design (SMACD)*, 2022, pp. 1–4.
- [3] Y.-L. Chen, Y.-R. Hong, Y.-T. Sung, and K.-E. Chang, “Efficacy of simulation-based learning of electronics using visualization and manipulation,” *Educational Technology & Society*, vol. 14, no. 2, pp. 269–277, 2011.
- [4] M. F. Adnan, N. Ahmed, A. Shafiullah, M. S. Rahman, and G. Sarowar, “A two-stage CNN-based hand-drawn electrical and electronic circuit component recognition system,” *Soft Computing*, vol. 33, pp. 13 367–13 390, 2021.
- [5] B. Edwards and V. Chandran, “Machine recognition of hand-drawn circuit diagrams,” in *Proc. IEEE Int. Conf. on Acoustics, Speech, and Signal Processing (ICASSP)*, vol. 6, Nov 2000, pp. 3618–3621.
- [6] C. R. Kelly and J. M. Cole, “Digitizing images of electrical-circuit schematics,” *APL Machine Learning*, vol. 2, no. 1, p. 016109, Jan 2024.
- [7] Y. Yamakaji, H. Shouno, and K. Fukushima, “Circuit2graph: Circuits with graph neural networks,” *IEEE Access*, vol. 11, pp. 123 456–123 467, 2023.
- [8] J. Bayer, A. K. Roy, and A. Dengel, “Instance segmentation based graph extraction for handwritten circuit diagram images,” *arXiv preprint arXiv:2301.03155*, Jan 2023, available: <https://arxiv.org/abs/2301.03155>.
- [9] M. S. Abhinav, V. L. Shrimanth, P. N. Ganesh, P. V. Nishanth, and K. Bhargavi, “Electric circuit diagram detection, recognition and simulation,” *International Research Journal of Engineering and Technology (IRJET)*, vol. 7, no. 9, pp. 3221–3227, Sep 2020.
- [10] D. Hemker, J. Maalouly, H. Mathis, R. Klos, and E. Ramanan, “From schematics to netlists—electrical circuit analysis using deep-learning methods,” *Advances in Radio Science*, vol. 22, pp. 61–75, Nov 2024.
- [11] KiCad, “SPICE simulation,” <https://www.kicad.org/discover/spice/>, 2024, accessed: 2025-01-15.

- [12] D. Vungarala, S. Alam, A. Ghosh, and S. Angizi, “SPICEPilot: Navigating SPICE code generation and simulation with AI guidance,” *arXiv preprint arXiv:2410.20553*, Oct 2024, available: <https://arxiv.org/abs/2410.20553>.